

Curso: APIs HTTP em Go

Oct 7, 2026 · @Daniel Freire

Como usar este curso

Você vai sair daqui sabendo construir, em Go 1.27, uma API HTTP de produção usando principalmente a biblioteca padrão: contrato em OpenAPI, erros padronizados, validação, segurança, testes de integração reais e observabilidade, organizada para continuar legível com centenas de rotas.

É o equivalente direto do curso de Minimal APIs no ASP.NET Core 10: os mesmos onze módulos, o mesmo método **top-down** (o que a API precisa garantir, depois o caminho de uma requisição, depois cada peça, e só então o código) e o mesmo projeto final. O que muda é a filosofia da plataforma: onde o ASP.NET Core oferece um framework que infere, injeta e gera, Go oferece peças pequenas e explícitas que você compõe. Cada módulo mostra as duas coisas: como fazer em Go e por que é diferente.

Pré-requisitos

Sintaxe básica de Go: tipos, structs, métodos, interfaces, slices, maps, goroutines e canais. Se ainda não a tem, o [Tour of Go](#) leva uma tarde. Experiência com APIs em qualquer linguagem ajuda; quem fez o curso de .NET vai reconhecer todos os problemas, só com outras soluções.

Exemplo condutor

A mesma **API de cobrança**: recebe uploads de arquivos de retorno, informa o status de cada arquivo, consulta títulos e ocorrências e transmite o progresso de um processamento em tempo real. Ela lê e grava as mesmas tabelas do pipeline de arquivos do primeiro curso (a tabela `arquivo`, a `retorno_detalhe`, a `outbox`). Uma API em Go e workers em .NET compartilhando o mesmo banco é um cenário real e comum.

Stack usada

- Go 1.27, `net/http` (servidor e roteamento), `log/slog`, `encoding/json` e o novo pacote `uuid` da biblioteca padrão
- PostgreSQL com `pgx` (driver) e `sqlc` (geração de código a partir de SQL)
- `oapi-codegen` para o contrato OpenAPI
- `golang.org/x/time/rate` para rate limiting e uma biblioteca de JWT com JWKS
- `net/http/httptest` e `testcontainers-go` para testes

- OpenTelemetry para Go (`otelhttp`)

Para quem vem do .NET

Conceito	ASP.NET Core 10	Go 1.27
Servidor	Kestrel	<code>http.Server</code> da biblioteca padrão
Rotas	<code>MapGet</code> , <code>MapGroup</code>	<code>http.ServeMux</code> com padrões como <code>GET /v1/arquivos/{id}</code>
Middleware	<code>app.Use(...)</code>	Função <code>func(http.Handler)</code> <code>http.Handler</code>
Binding	Inferido pelos parâmetros do handler	Explícito: <code>r.PathValue</code> , <code>r.URL.Query()</code> , decodificação de JSON
Respostas tipadas	<code>TypedResults</code> , <code>Results<...></code>	Funções auxiliares e tipos de erro seus
Erros	Exceções + <code>ExceptionHandler</code>	Erros como valores de retorno
Injeção de dependência	Contêiner de DI	Structs com campos, montadas à mão no <code>main</code>
Cancelamento	<code>CancellationToken</code>	<code>context.Context (r.Context())</code>
OpenAPI	Gerado do código	Contrato escrito primeiro, código gerado a partir dele
Rate limiting	<code>AddRateLimiter</code>	<code>golang.org/x/time/rate</code>
Testes de integração	<code>WebApplicationFactory</code>	<code>httptest</code>

A tabela resume o curso inteiro: quase tudo que o .NET faz por convenção, em Go você escreve em poucas linhas óbvias. O custo é mais código; o ganho é que não há nada escondido.

Mapa dos módulos

Módulo	Pergunta que responde	O que você entrega
1. O problema	O que a API garante, e o que muda em Go?	Requisitos e a escolha entre biblioteca padrão e framework

Módulo	Pergunta que responde	O que você entrega
2. Arquitetura	Que caminho uma requisição percorre?	O mapa do servidor, do mux e da cadeia de middlewares
3. Fundamentos	Como declaro rotas e leio a entrada?	Os primeiros handlers, com JSON e tipos de domínio
4. Respostas e erros	Como devolvo sucesso e falha de forma previsível?	ProblemDetails, erros como valores e validação
5. Organização	Como o código cresce sem virar um <code>main.go</code> gigante?	Pacotes por feature e dependências explícitas
6. Dados	Como ler, gravar e receber arquivos sem travar nada?	pgx, sqlc, keyset, streaming e upload
7. Contrato	Como o cliente sabe o que a API faz?	OpenAPI como fonte da verdade
8. Segurança	Quem pode chamar o quê, e quantas vezes?	JWT, autorização, rate limiting e CORS
9. Concorrência e performance	Como responder rápido sob carga?	Timeouts, cache, SSE e profiling
10. Testes	Como provo que tudo funciona?	Testes de tabela, httptest e banco real
11. Operação	Como sei que está saudável em produção?	Shutdown gracioso, telemetria e deploy
Projeto final	Tudo junto funciona?	A API de cobrança completa em Go

Sugestão de ritmo: um módulo por semana. Os Módulos 2, 4 e 5 concentram o que mais difere do .NET e merecem mais tempo.

Módulo 1 — O problema: o que a API garante, e o que muda em Go

As garantias de uma API de produção não dependem da linguagem: contrato explícito, erros previsíveis, entrada validada, autorização por recurso, idempotência, limites, desempenho, evolução e operabilidade. O que Go muda é **quem** cumpre cada garantia: em vez de um framework que faz por convenção, é código seu, pequeno e visível. Este módulo fixa as garantias e explica as diferenças de Go que mudam o jeito de escrever uma API.

1.1 As garantias, e onde cada uma mora em Go

Garantia	Pergunta que ela responde	Onde o curso trata	Em Go, quem cumpre
Contrato explícito	O cliente sabe o que mandar e o que recebe?	Módulo 7	A especificação OpenAPI, com código gerado dela
Erros previsíveis	Toda falha tem o mesmo formato?	Módulo 4	Uma função única que escreve ProblemDetails
Entrada validada	Dado inválido é barrado antes da regra?	Módulo 4	Métodos <code>Validar()</code> nos tipos de entrada
Autorização	Cada cliente só vê o que é dele?	Módulo 8	Middleware de autenticação + filtro por cliente em toda consulta
Idempotência	Repetir uma chamada duplica efeito?	Módulo 6	Tabela de chaves e <code>ON CONFLICT</code>
Limites	Um cliente pode derrubar os outros?	Módulos 8 e 9	Rate limiter por cliente e timeouts do servidor
Desempenho	A latência aguenta o pico?	Módulos 6 e 9	Consultas certas, streaming, cache
Evolução	Dá para mudar sem quebrar clientes?	Módulo 7	Diff da especificação no CI
Operabilidade	Dá para rastrear uma requisição?	Módulo 11	<code>slog</code> + OpenTelemetry com <code>trace_id</code>

A semântica de HTTP (verbos seguros e idempotentes, os status codes certos, `Idempotency-Key` para `POST`) é exatamente a mesma do curso de .NET, Módulo 1.2. Se ainda não leu, leia aquela seção antes de seguir: ela vale palavra por palavra aqui.

1.2 Cinco diferenças de Go que mudam a forma de uma API

1. Erros são valores, não exceções. Toda função que pode falhar devolve um `error` como último resultado, e quem chama decide o que fazer ali mesmo. Não existe um `try/catch` no topo que transforme falhas em 500; cada handler trata cada erro, e o Módulo 4 mostra como fazer isso sem repetição.

```

arquivo, err := repo.Obter(ctx, clienteID, id)
if errors.Is(err, ErrNaoEncontrado) {
    escreverProblema(w, r, http.StatusNotFound, "arquivo-nao-encontrado",
"Arquivo não encontrado.")
    return
}
if err != nil {
    escreverErroInterno(w, r, err)
    return
}

```

`panic` existe, mas é para bugs (índice fora do slice, ponteiro nulo), não para fluxo de negócio. O servidor HTTP do Go recupera um `panic` dentro de um handler e fecha só aquela conexão, mas sem resposta útil; o Módulo 4 coloca um middleware de recuperação que devolve um 500 em `ProblemDetails`.

2. Interfaces são implícitas e pequenas. Um tipo implementa uma interface só por ter os métodos, sem declarar `: IRepository`. O idioma de Go é definir a interface **no pacote que a usa**, com só os métodos de que ele precisa. Isso torna testes triviais: o handler pede uma interface de um método, e o teste passa uma struct falsa de três linhas.

3. Não há contêiner de injeção de dependência. Dependências são campos de structs, preenchidos à mão no `main`. Parece primitivo e é libertador: o grafo de dependências inteiro está em uma função que você lê de cima para baixo, e o compilador garante que nada falta. Não existe "dependência cativa" (Módulo 2 do curso de .NET) porque não existem tempos de vida automáticos: você decide e vê.

4. Cada requisição roda na sua goroutine, e o `context.Context` é o fio que a amarra. O servidor cria uma goroutine por requisição (barata: alguns KB de pilha). O `r.Context()` é cancelado quando o cliente desiste ou o timeout estoura, e deve ser passado como **primeiro parâmetro** de toda função que faz I/O. É o `CancellationToken` do .NET, mais o lugar onde moram valores da requisição, como o usuário autenticado.

5. Valores zero são úteis, e `nil` é uma decisão de projeto. Todo tipo tem um valor zero (`0`, `""`, `false`, `nil`). Na decodificação de JSON, um campo ausente vira o valor zero, então "não enviado" e "enviado como zero" são indistinguíveis, a menos que o campo seja ponteiro (`*int`). Isso afeta validação e contrato, e o Módulo 3 trata disso.

1.3 Biblioteca padrão ou framework?

Até o Go 1.21, o `http.ServeMux` padrão não distinguia métodos HTTP nem tinha parâmetros de rota, e quase todo projeto usava um roteador externo. Desde o Go 1.22, ele aceita padrões como `GET /v1/arquivos/{id}`, e a biblioteca padrão passou a bastar para a maioria das APIs.

Opção	O que é	Compatível com net/http	Quando escolher
net/http puro	Servidor, mux com método e curingas, middleware por composição	É a própria	Padrão para APIs novas; este curso
chi	Roteador leve com sub-roteadores e middlewares prontos	Sim, totalmente	Quando quiser grupos de rotas com middleware sem escrever o encadeamento à mão
Gin / Echo	Frameworks com contexto próprio, binding e validação embutidos	Parcial (handlers têm assinatura própria)	Times que querem a experiência mais próxima de um framework web
Fiber	Framework sobre fasthttp, não sobre net/http	Não	Raramente: perde o ecossistema de middlewares e instrumentação padrão
Huma	Camada sobre net/http que gera OpenAPI a partir de tipos Go	Sim	Quando o time prefere contrato gerado do código, como no .NET

O critério que mais pesa é a coluna do meio. Tudo que é compatível com `http.Handler` funciona com toda a instrumentação do ecossistema (OpenTelemetry, CORS, compressão, rate limiting) e com `httptest`. Por isso este curso usa a biblioteca padrão e mostra, onde fizer diferença, o que um roteador como o chi economizaria.

1.4 Os requisitos da API de cobrança

Os mesmos endpoints do curso de .NET, agora escritos como padrões do `ServeMux`:

Padrão de rota	O que faz	Detalhe que torna difícil
POST /v1/arquivos	Recebe um arquivo de retorno	Upload grande em streaming, idempotência, 202

Padrão de rota	O que faz	Detalhe que torna difícil
GET /v1/arquivos/{id}	Status do processamento	Autorização: só o dono vê
GET /v1/arquivos/{id}/progresso	Progresso em tempo real	Server-Sent Events
GET /v1/arquivos	Lista arquivos do cliente	Paginação keyset, filtros
GET /v1/titulos/{nossoNumero}	Consulta um título	Cache, 404 correto
GET /v1/titulos/{nossoNumero}/ocorrencias	Histórico do título	Lista potencialmente longa
GET /v1/ocorrencias/export	Exporta ocorrências do dia	Resposta em streaming de milhões de linhas

Requisitos não funcionais de exemplo, iguais aos do curso de .NET: p95 abaixo de 200 ms para consultas, uploads de até 500 MB, 100 requisições por minuto por cliente, JWT emitido por um provedor de identidade externo.

Exercício do Módulo 1

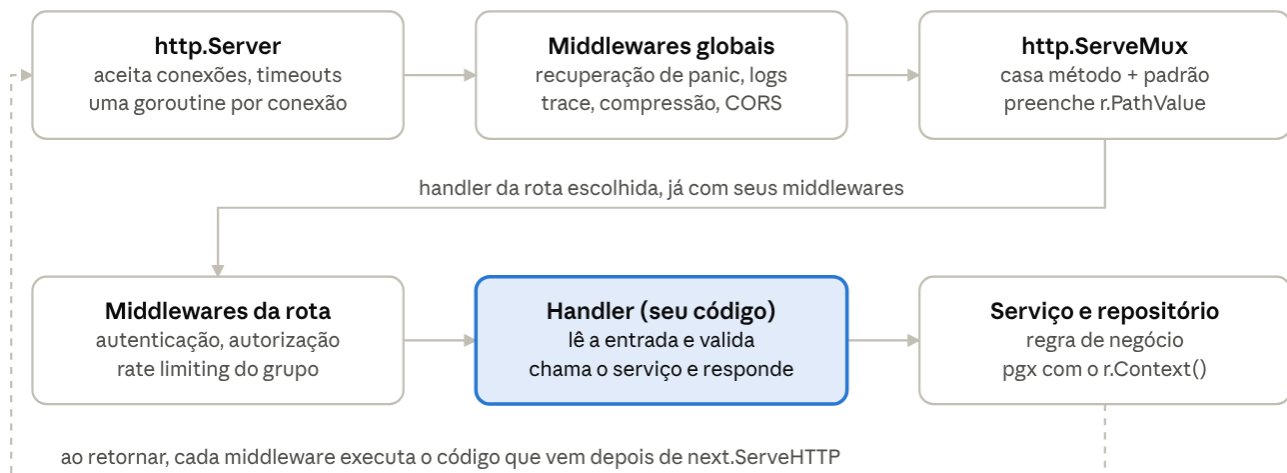
Escreva, para cada uma das cinco diferenças da seção 1.2, um exemplo de onde ela apareceria na API de cobrança (por exemplo: "diferença 5: o campo `limite` da listagem precisa distinguir ausente de zero"). Depois instale o Go 1.27, crie o módulo com `go mod init` e rode `go version` e `go env GOVERSION` para confirmar a versão.

Módulo 2 — Arquitetura: o caminho de uma requisição em Go

Em Go só existe uma abstração no caminho de uma requisição: a interface `http.Handler`, com um único método `ServeHTTP(w, r)`. O servidor chama um handler; esse handler é uma cadeia de middlewares que embrulham o `ServeMux`; o mux escolhe o handler da rota, que por sua vez pode estar embrulhado por middlewares próprios. Entender essa composição é entender o framework inteiro, porque não há mais nada.

Em Go, a requisição atravessa handlers que embrulham handlers

Caminho de uma requisição no `net/http` · tudo, do servidor ao seu código, é um `http.Handler`



caminho de uma requisição no `net/http` · 6 etapas

Compare com o pipeline do ASP.NET Core: a forma é a mesma (ida por camadas, volta na ordem inversa), mas em Go cada camada é uma função comum que você pode ler, e a ordem é a ordem em que as funções se embrulham.

2.1 `http.Server`: configure os timeouts, sempre

O `http.Server` com valores zero **não tem timeout nenhum**. Um cliente que abre uma conexão e envia um byte de header por minuto segura uma goroutine e um descritor de arquivo para sempre (o ataque conhecido como Slowloris). Toda API de produção define os timeouts explicitamente:

```

srv := &http.Server{
    Addr:           ":8080",
    Handler:        raiz,                // a cadeia de middlewares com o
mux dentro
    ReadHeaderTimeout: 5 * time.Second, // tempo para receber os headers
    ReadTimeout:       30 * time.Second, // headers + corpo inteiro
    WriteTimeout:      30 * time.Second, // do fim da leitura dos headers
    até o fim da resposta
    IdleTimeout:       120 * time.Second, // conexão keep-alive ociosa
    MaxHeaderBytes:    64 << 10,         // 64 KiB de headers
}
  
```

Esses valores valem para todas as requisições, e três rotas da API precisam de mais: o upload de 500 MB (o `ReadTimeout` cortaria o corpo no meio), a exportação e o stream de progresso (o `WriteTimeout` cortaria a resposta). A solução é ajustar o prazo **por requisição** com `http.ResponseWriter`, dentro do handler daquela rota:

```
rc := http.NewResponseController(w)
_ = rc.SetReadDeadline(time.Now().Add(15 * time.Minute)) // só nesta
requisição
_ = rc.SetWriteDeadline(time.Time{}) // zero = sem prazo
de escrita
```

2.2 Handler e ResponseWriter

Um handler recebe um `http.ResponseWriter` (para escrever a resposta) e um `*http.Request` (a requisição). `http.HandlerFunc` adapta uma função comum para a interface, então na prática você escreve funções. Três regras do `ResponseWriter` causam a maioria dos bugs de quem chega:

1. **Headers antes de tudo.** `w.Header().Set(...)` só tem efeito antes da primeira chamada a `WriteHeader` ou `Write`. Depois disso os headers já foram enviados.
2. **WriteHeader uma vez só.** A segunda chamada é ignorada e gera um aviso no log ("superfluous WriteHeader call"). Se você chamar `Write` sem `WriteHeader`, o status é 200 implicitamente.
3. **Escrever não encerra o handler.** Depois de escrever um erro, é preciso `return`. Esquecer o `return` faz o handler seguir em frente e escrever outra resposta por cima.

2.3 Middleware: uma função que embrulha um handler

Um middleware em Go é uma função `func(http.Handler) http.Handler`: recebe o próximo handler e devolve um novo que faz algo antes, chama o próximo, e faz algo depois. Este registra cada requisição com o status de resposta:

```
type gravadorDeStatus struct {
    http.ResponseWriter
    status int
}

func (g *gravadorDeStatus) WriteHeader(status int) {
    g.status = status
    g.ResponseWriter.WriteHeader(status)
}

func (g *gravadorDeStatus) Write(b []byte) (int, error) {
    if g.status == 0 {
        g.status = http.StatusOK
    }
    return g.ResponseWriter.Write(b)
}
```

```
// Unwrap deixa o http.ResponseWriter alcançar o writer original (Flush,
prazos)
func (g *gravadorDeStatus) Unwrap() http.ResponseWriter { return
g.ResponseWriter }

func ComLog(log *slog.Logger) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request)
{
            inicio := time.Now()
            g := &gravadorDeStatus{ResponseWriter: w}

            next.ServeHTTP(g, r) // todo o resto da cadeia

            log.LogAttrs(r.Context(), slog.LevelInfo, "requisição",
                slog.String("metodo", r.Method),
                slog.String("rota", r.Pattern), // o padrão que casou, não a
URL com IDs
                slog.Int("status", g.status),
                slog.Duration("duracao", time.Since(inicio)))
            })
        })
    }
}
```

O método `Unwrap` merece atenção. Embrulhar o `ResponseWriter` esconde as interfaces opcionais do writer original, como `http.Flusher`, necessária para SSE. O `http.ResponseController` procura um método `Unwrap` para chegar ao writer verdadeiro; sem ele, o stream de progresso do Módulo 9 deixa de funcionar assim que você adiciona o log. O campo `r.Pattern` contém o padrão de rota que casou, como `GET /v1/arquivos/{id}`, e é o que deve ir para logs e métricas (baixa cardinalidade).

Para aplicar vários middlewares numa ordem legível, uma função de encadeamento resolve:

```
// Encadear(h, a, b, c) == a(b(c(h))): o primeiro da lista é o mais externo
func Encadear(h http.Handler, mws ...func(http.Handler) http.Handler)
http.Handler {
    for i := len(mws) - 1; i >= 0; i-- {
        h = mws[i](h)
    }
    return h
}

raiz := Encadear(mux,
    ComRecuperacao(log), // 1. pega panics de tudo que vem depois
    ComTrace,           // 2. abre o span da requisição
    ComLog(log),         // 3. registra status e duração
    ComCompressao,       // 4. comprime a resposta
)
```

2.4 ServeMux: padrões com método e curingas

Desde o Go 1.22, cada padrão registrado no mux pode ter método, host e curingas:

```
mux.HandleFunc("GET /v1/arquivos/{id}", obterArquivo)
mux.HandleFunc("POST /v1/arquivos", receberArquivo)
mux.HandleFunc("GET /v1/arquivos/{id}/progresso", progresso)
mux.HandleFunc("GET /v1/{$$}", indice) // {$$: só /v1/ exatamente
mux.HandleFunc("GET /arquivos-brutos/{chave...}", baixar) // {chave...}: o
resto do caminho
```

As regras que importam:

- **r.PathValue("id")** devolve o valor do curinga como `string`. A conversão e a validação são suas (Módulo 3).
- **O padrão mais específico vence**, independentemente da ordem de registro. `GET /v1/arquivos/{id}/progresso` vence `GET /v1/arquivos/{id}` para essa URL. Dois padrões que se sobrepõem sem que um seja mais específico causam `panic` no registro, ou seja, na subida da aplicação: um conflito nunca chega à produção em silêncio.
- **Método errado devolve 405** com o header `Allow` preenchido, automaticamente. Um padrão com `GET` também atende `HEAD`.
- **Barra final importa.** `/v1/arquivos/` (com barra) casa toda a subárvore; `/v1/arquivos` (sem barra) casa só aquele caminho. Prefira sempre padrões sem barra final, e `{$}` quando precisar da raiz exata.

2.5 Grupos de rotas sem MapGroup

O mux padrão não tem grupos com middleware, mas um tipo de vinte linhas faz o papel do `MapGroup`:

```
type Middleware = func(http.Handler) http.Handler

type Grupo struct {
    mux      *http.ServeMux
    prefixo  string
    mws      []Middleware
}

func NovoGrupo(mux *http.ServeMux, prefixo string, mws ...Middleware) Grupo {
    return Grupo{mux: mux, prefixo: prefixo, mws: mws}
}

// Sub cria um grupo filho que herda prefixo e middlewares
func (g Grupo) Sub(prefixo string, mws ...Middleware) Grupo {
    return Grupo{mux: g.mux, prefixo: g.prefixo + prefixo, mws:
        append(slices.Clone(g.mws), mws...)}
}

// Rota registra "MÉTODO /prefixo/caminho" com os middlewares do grupo e os
da rota
func (g Grupo) Rota(metodo, caminho string, h http.Handler, mws
...Middleware) {
    todos := append(slices.Clone(g.mws), mws...)
    g.mux.Handle(metodo+" "+g.prefixo+caminho, Encadear(h, todos...))
}
```

O `slices.Clone` não é detalhe: sem ele, dois subgrupos criados a partir do mesmo grupo poderiam compartilhar o mesmo array por baixo, e o `append` de um sobrescreveria os middlewares do outro. Se preferir não manter esse código, o roteador `chi` oferece exatamente isso (`r.Route`, `r.With`, `r.Group`) e continua compatível com `http.Handler`.

2.6 context.Context: cancelamento e valores da requisição

O `r.Context()` carrega três coisas: o cancelamento (o cliente fechou a conexão), um prazo opcional e valores associados à requisição. Para valores, use uma chave de tipo próprio e não exportado, que nenhum outro pacote consegue colidir, e funções tipadas para ler e gravar:

```

type chaveUsuario struct{}

func ComUsuario(ctx context.Context, u Usuario) context.Context {
    return ctx.WithValue(ctx, chaveUsuario{}, u)
}

func UsuarioDe(ctx context.Context) (Usuario, bool) {
    u, ok := ctx.Value(chaveUsuario{}).(Usuario)
    return u, ok
}

```

O middleware de autenticação (Módulo 8) chama `ComUsuario` e passa adiante `r.WithContext(ctx)`; os handlers chamam `UsuarioDe`. Use valores de contexto só para dados que realmente pertencem à requisição (usuário, trace, ID de correlação), nunca para dependências como o pool do banco; essas são campos de struct (Módulo 5).

2.7 Tudo roda em paralelo

Cada requisição é uma goroutine, e não há escopo por requisição como no DI do .NET. Qualquer estado que os handlers compartilham (um cache em memória, um contador, um mapa de limitadores por cliente) precisa de proteção com `sync.Mutex`, `sync.RWMutex`, `sync.Map` ou tipos de `sync/atomic`. O pool do `pgx` (`pgxpool.Pool`) e o `*slog.Logger` já são seguros para uso concorrente. Rode os testes sempre com `go test -race`: o detector de corrida encontra acesso concorrente sem proteção que nenhum teste funcional pegaria.

Exercício do Módulo 2

Monte um servidor com o `http.Server` configurado como em 2.1, a cadeia `ComRecuperacao + ComLog` e duas rotas: `GET /v1/arquivos/{id}` e `GET /v1/arquivos/{id}/progresso`. Confirme no log que `r.Pattern` aparece com o padrão, não com o ID. Faça um `POST` numa rota só de `GET` e veja o 405 com `Allow`. Por fim, registre dois padrões conflitantes (`GET /v1/{x}/a` e `GET /v1/b/{y}`) e observe o `panic` na subida, com a mensagem que explica o conflito.

Módulo 3 — Fundamentos: rotas, entrada e JSON

Em Go não há model binding: o handler lê cada dado da requisição explicitamente, converte para um tipo do domínio e decide o que fazer se a conversão falhar. Isso parece trabalho a mais, mas dá controle total sobre o contrato. O truque para não repetir código é concentrar a conversão em funções de parse dos próprios tipos de domínio e numa função genérica de leitura de JSON.

3.1 O main que se testa: o padrão run

O `main` faz o mínimo: monta o contexto que é cancelado no `SIGTERM` e chama uma função `run` que devolve erro. Toda a lógica de subida fica em `run`, que recebe suas dependências externas (variáveis de ambiente, saída de log) como parâmetros e, por isso, pode ser chamada de um teste:

```
func main() {
    ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt,
    syscall.SIGTERM)
    defer stop()

    if err := run(ctx, os.Getenv, os.Stdout); err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }
}

func run(ctx context.Context, getenv func(string) string, saida io.Writer)
error {
    cfg, err := carregarConfig(getenv)
    if err != nil {
        return fmt.Errorf("configuração: %w", err)
    }
    log := slog.New(slog.NewJSONHandler(saida, nil))

    mux := http.NewServeMux()
    mux.HandleFunc("GET /v1/titulos/{nossoNumero}", obterTitulo)

    srv := &http.Server{Addr: cfg.Endereco, Handler: mux, ReadHeaderTimeout:
5 * time.Second}
    log.Info("servidor ouvindo", "endereco", cfg.Endereco)
    return srv.ListenAndServe() // o desligamento gracioso entra no Módulo 11
}
```

`fmt.Errorf("configuração: %w", err)` é o idioma para adicionar contexto a um erro sem perder o original: o `%w` o embrulha, e `errors.Is / errors.As` continuam enxergando-o. A configuração fica numa struct validada na subida:

```
type Config struct {
    Endereco      string
    BancoURL      string
    JWTEmissor    string
    JWTAudiencia  string
}
```

```
func carregarConfig(getenv func(string) string) (Config, error) {
    cfg := Config{
        Endereco:      cmp.Or(getenv("ENDERECO"), ":8080"), // cmp.Or:
primeiro valor não zero
        BancoURL:      getenv("BANCO_URL"),
        JWTEmissor:      getenv("JWT_EMISSOR"),
        JWTAudiencia:    getenv("JWT_AUDIENCIA"),
    }
    var faltando []string
    if cfg.BancoURL == "" {
        faltando = append(faltando, "BANCO_URL")
    }
    if cfg.JWTEmissor == "" || cfg.JWTAudiencia == "" {
        faltando = append(faltando, "JWT_EMISSOR/JWT_AUDIENCIA")
    }
    if len(faltando) > 0 {
        return Config{}, fmt.Errorf("variáveis obrigatórias ausentes: %s",
strings.Join(faltando, ", "))
    }
    return cfg, nil
}
```

É o equivalente ao `ValidateOnStart` do .NET: configuração errada impede a subida, não quebra a primeira requisição.

3.2 Lendo cada parte da requisição

Origem	Como ler	Observação
Curinga de rota	<code>r.PathValue("id")</code>	Sempre <code>string</code> ; vazio se o nome não existe no padrão
Query string	<code>r.URL.Query().Get("limite")</code>	<code>Query()</code> analisa a URL a cada chamada; guarde o resultado numa variável
Ausência na query	<code>q.Has("limite")</code>	Distingue "não enviado" de "enviado vazio"
Header	<code>r.Header.Get("Idempotency-Key")</code>	Nomes são normalizados; a busca ignora maiúsculas
Corpo JSON	<code>json.UnmarshalRead(r.Body, &v)</code>	Sempre com limite de tamanho (3.4)
Corpo cru	<code>r.Body</code> (um <code>io.Reader</code>)	Upload em streaming (Módulo 6)

Origem	Como ler	Observação
Usuário	<code>UsuarioDe(r.Context())</code>	Colocado pelo middleware de autenticação

Para a própria biblioteca padrão, um ID de arquivo é só uma string. Quem decide que ele é um UUID é você, com o novo pacote `uuid` do Go 1.27:

```
id, err := uuid.Parse(r.PathValue("id"))
if err != nil {
    http.Error(w, "id inválido", http.StatusBadRequest) // vira
    ProblemDetails no Módulo 4
    return
}
```

Uma diferença de contrato em relação ao .NET: lá, a restrição `{id:guid}` fazia um ID inválido virar 404 (a rota não casava). Aqui a rota sempre casa e você escolhe a resposta; 400 com uma mensagem clara costuma ser mais útil para quem integra.

3.3 Tipos de domínio com parse

O equivalente ao `NossoNumero : IParsable` do .NET é um tipo definido mais uma função de parse que é o único jeito de construir um valor válido a partir de texto:

```
package dominio

var ErrNossoNumeroInvalido = errors.New("nosso número deve ter de 1 a 18
dígitos e não ser zero")

type NossoNumero int64

func ParseNossoNumero(s string) (NossoNumero, error) {
    if s == "" || len(s) > 18 { // 18 dígitos sempre cabem em int64, sem
    risco de estouro
        return 0, ErrNossoNumeroInvalido
    }
    var v int64
    for i := 0; i < len(s); i++ {
        c := s[i]
        if c < '0' || c > '9' {
            return 0, ErrNossoNumeroInvalido
        }
        v = v*10 + int64(c-'0')
    }
    if v == 0 {
```

```

        return 0, ErrNossoNumeroInvalido
    }
    return NossoNumero(v), nil
}

// UnmarshalText faz o mesmo parse valer quando o tipo aparece num corpo JSON
func (n *NossoNumero) UnmarshalText(b []byte) error {
    v, err := ParseNossoNumero(string(b))
    if err != nil {
        return err
    }
    *n = v
    return nil
}

func (n NossoNumero) MarshalText() ([]byte, error) {
    return strconv.AppendInt(nil, int64(n), 10), nil
}

```

Com `MarshalText` / `UnmarshalText`, o tipo vira uma string no JSON e passa pela mesma validação em qualquer origem: rota, query ou corpo. O valor de status do arquivo segue a mesma ideia, com um tipo string e constantes que são exatamente os valores gravados na tabela `arquivo`:

```

type StatusArquivo string

const (
    StatusRecebido    StatusArquivo = "RECEBIDO"
    StatusProcessando StatusArquivo = "PROCESSANDO"
    StatusConcluido   StatusArquivo = "CONCLUIDO"
    StatusRejeitado   StatusArquivo = "REJEITADO"
    StatusFalhou      StatusArquivo = "FALHOU"
)

func (s StatusArquivo) Valido() bool {
    switch s {
    case StatusRecebido, StatusProcessando, StatusConcluido, StatusRejeitado,
    StatusFalhou:
        return true
    }
    return false
}

```

3.4 Lendo JSON com segurança: encoding/json/v2

O Go 1.27 trouxe o `encoding/json/v2` para a biblioteca padrão, e o `encoding/json` clássico passou a usar a mesma implementação por baixo. O v2 tem padrões mais rígidos e interoperáveis: rejeita UTF-8 inválido e nomes duplicados num objeto, e compara nomes de campo diferenciando maiúsculas (o v1 aceitava `"Banco"` para o campo `banco`). Para APIs novas, use o v2.

Toda leitura de corpo passa por uma função genérica que impõe limite de tamanho e recusa campos desconhecidos:

```
import (
    "encoding/json/v2"
    "net/http"
)

// lerJSON decodifica o corpo em T, com no máximo 'limite' bytes e sem campos
desconhecidos
func lerJSON[T any](w http.ResponseWriter, r *http.Request, limite int64) (T,
error) {
    var v T
    corpo := http.MaxBytesReader(w, r.Body, limite)
    err := json.UnmarshalRead(corpo, &v, json.RejectUnknownMembers(true))
    return v, err
}
```

Por que cada peça:

- **`http.MaxBytesReader`** devolve erro quando o corpo passa do limite e avisa o servidor para fechar a conexão. Sem ele, um cliente pode enviar gigabytes num endpoint que esperava 2 KB. O erro é do tipo `*http.MaxBytesError`, que o Módulo 4 converte em 413.
- **`RejectUnknownMembers`** faz um campo com nome errado (`"ur11"` em vez de `"url"`) virar erro em vez de ser ignorado em silêncio. Para APIs públicas que evoluem, alguns times preferem tolerar campos desconhecidos; escolha conscientemente e documente.
- **Genérico:** o tipo de destino vem do chamador, `pedido, err := lerJSON[RegistrarCallbackRequest](w, r, 16<<10)`, sem conversões.

3.5 Filtros de listagem: ausente, vazio e zero

A diferença 5 do Módulo 1 aparece aqui. Um filtro de listagem precisa saber se `limite` não foi enviado (usar 50), foi enviado vazio ou foi enviado como `abc` (erro):

```

type FiltroDeArquivos struct {
    Status *dominio.StatusArquivo // nil = sem filtro
    De     *time.Time
    Ate     *time.Time
    Cursor string
    Limite  int
}

// lerFiltro converte a query e acumula um erro por campo (o Módulo 4
transforma isso em 400)
func lerFiltro(q url.Values) (FiltroDeArquivos, ErrosDeCampo) {
    f := FiltroDeArquivos{Limite: 50, Cursor: q.Get("cursor")}
    erros := ErrosDeCampo{}

    if q.Has("limite") {
        n, err := strconv.Atoi(q.Get("limite"))
        if err != nil {
            erros.Add("limite", "deve ser um número inteiro")
        } else {
            f.Limite = n
        }
    }

    if s := q.Get("status"); s != "" {
        st := dominio.StatusArquivo(s)
        if !st.Valido() {
            erros.Add("status", "valor desconhecido")
        }
        f.Status = &st
    }

    f.De = lerData(q, "de", erros)
    f.Ate = lerData(q, "ate", erros)
    return f, erros
}

func lerData(q url.Values, campo string, erros ErrosDeCampo) *time.Time {
    s := q.Get(campo)
    if s == "" {
        return nil
    }

    t, err := time.Parse(time.DateOnly, s) // "2006-01-02"
    if err != nil {
        erros.Add(campo, "use o formato AAAA-MM-DD")
        return nil
    }
    return &t
}

```

```
// ErrosDeCampo acumula mensagens por nome de campo
type ErrosDeCampo map[string][]string

func (e ErrosDeCampo) Add(campo, msg string) { e[campo] = append(e[campo],
msg) }
```

Ponteiros (`*time.Time`, `*StatusArquivo`) representam "opcional". Go não tem `Nullable<T>`; ponteiro é a forma idiomática, e o `new(expressão)` do Go 1.26 deixa isso mais confortável ao montar valores: De: `new(time.Now())` em vez de declarar uma variável só para tirar o endereço.

3.6 JSON de saída: tags definem o contrato

Os nomes no JSON vêm das *struct tags*, não dos nomes dos campos. Escreva-as sempre, mesmo quando parecem óbvias: renomear um campo Go sem tag mudaria o contrato em silêncio.

```
type ArquivoResposta struct {
    ID                uuid.UUID          `json:"id"`
    NomeOriginal      string             `json:"nomeOriginal"`
    Banco             string             `json:"banco"`
    Status            dominio.StatusArquivo `json:"status"`
    RecebidoEm        time.Time          `json:"recebidoEm"`
    LinhasProcessadas int                `json:"linhasProcessadas"`
    LinhasRejeitadas  int                `json:"linhasRejeitadas"`
    Motivo            string             `json:"motivo,omitzero"`
}
```

`uuid.UUID` e `time.Time` já se serializam como texto (UUID em minúsculas com hífen, data em RFC 3339). A opção `omitzero` omite o campo quando ele tem o valor zero do tipo; aqui, `motivo` só aparece quando o arquivo foi rejeitado ou falhou. A tabela de entidade do banco nunca é serializada direto: a struct de resposta é o contrato, e a do banco pode mudar sem quebrá-lo.

3.7 Um handler completo

Juntando as peças, eis o primeiro handler de verdade (ainda com `http.Error`, que o Módulo 4 substitui):

```
func obterTitulo(w http.ResponseWriter, r *http.Request) {
    numero, err := dominio.ParseNossoNumero(r.PathValue("nossoNumero"))
    if err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }
}
```

```

    }

    titulo, err := buscarTitulo(r.Context(), numero) // o contexto vai junto,
    sempre
    if errors.Is(err, dominio.ErrNaoEncontrado) {
        http.Error(w, "título não encontrado", http.StatusNotFound)
        return
    }
    if err != nil {
        http.Error(w, "erro interno", http.StatusInternalServerError)
        return
    }

    w.Header().Set("Content-Type", "application/json")
    _ = json.MarshalWrite(w, titulo)
}

```

Repare na ordem: headers, depois o corpo, e um `return` depois de cada resposta de erro. E repare no erro de `MarshalWrite` ignorado: quando ele falha, a resposta já começou e o cliente provavelmente desconectou, então não há o que responder; o certo é registrar em log, como o Módulo 4 faz.

Exercício do Módulo 3

Implemente `GET /v1/titulos/{nossoNumero}` com um repositório em memória e teste quatro chamadas: número válido existente (200), válido inexistente (404), `abc` (400) e `0` (400). Depois implemente `lerFiltro` e confira que `?limite=x` gera erro, `?limite=` também, e a ausência de `limite` usa 50. Por fim, envie um corpo JSON de 1 MB para um endpoint com `lerJSON` limitado a 16 KB e observe o erro `*http.MaxBytesError`.

Módulo 4 — Respostas e erros previsíveis

O problema que o `IExceptionHandler` resolve no .NET (transformar qualquer falha numa resposta padronizada, num lugar só) existe igual em Go, mas sem exceções. A solução idiomática é fazer os handlers **devolverem erro**, e um adaptador único converter esse erro em `ProblemDetails`. Erros de negócio viram tipos de erro com código e status; validação vira um erro que carrega as mensagens por campo.

4.1 Handlers que devolvem erro

A assinatura padrão `func(w, r)` não tem retorno, o que obriga cada handler a escrever a própria resposta de erro. Um tipo de quatro linhas muda isso:

```
// Handler é um handler que pode falhar; o erro vira ProblemDetails num lugar só
type Handler func(w http.ResponseWriter, r *http.Request) error

func (h Handler) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    if err := h(w, r); err != nil {
        problema.DeErro(w, r, err)
    }
}
```

Como `Handler` implementa `http.Handler`, ele entra no mux e na cadeia de middlewares como qualquer outro: `mux.Handle("GET /v1/arquivos/{id}", Handler(h.obterArquivo))`. O handler fica com um formato único, "ou responde, ou devolve um erro":

```
func (h *Arquivos) obterArquivo(w http.ResponseWriter, r *http.Request) error {
    id, err := uuid.Parse(r.PathValue("id"))
    if err != nil {
        return validacao.Campo("id", "deve ser um UUID")
    }
    usuario := auth.Usuario(r.Context())

    arquivo, err := h.consulta.Obter(r.Context(), usuario.ClienteID, id)
    if err != nil {
        return err // dominio.ErrNaoEncontrado vira 404; o resto vira 500
    }
    return resposta.JSON(w, http.StatusOK, arquivo)
}
```

Regra de ouro: depois que o handler começou a escrever a resposta, ele não pode mais devolver erro, porque o adaptador tentaria escrever uma segunda resposta. Por isso o helper de JSON serializa primeiro e só escreve quando tudo deu certo:

```
func JSON(w http.ResponseWriter, status int, v any) error {
    corpo, err := json.Marshal(v)
    if err != nil {
        return fmt.Errorf("serializar resposta: %w", err) // nada foi
escrito: o adaptador pode responder 500
    }
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    _, _ = w.Write(corpo) // se falhar aqui, o cliente desconectou: não há a
quem responder
    return nil
}
```

Respostas grandes ou em streaming (Módulo 6) não usam esse helper, justamente porque não dá para montar tudo antes.

4.2 ProblemDetails em Go

O formato é o da RFC 9457, idêntico ao do curso de .NET, para que um cliente que integra com as duas APIs trate erros do mesmo jeito:

```
package problema

const BaseTipos = "https://api.cobranca.exemplo/erros/"

type Detalhes struct {
    Type      string          `json:"type"`
    Title     string          `json:"title"`
    Status    int             `json:"status"`
    Detail    string          `json:"detail,omitzero"`
    Instance  string          `json:"instance,omitzero"`
    TraceID   string          `json:"traceId,omitzero"`
    Errors    map[string][]string `json:"errors,omitzero"`
    Extensoes map[string]any    `json:",embed" // campos extras no mesmo
nível (ex.: arquivoExistenteId)
}

func Escrever(w http.ResponseWriter, r *http.Request, d Detalhes) {
    if d.Type == "" {
        d.Type = "about:blank"
    }
    d.Instance = r.Method + " " + r.URL.Path
    if sc := trace.SpanContextFromContext(r.Context()); sc.HasTraceID() {
        d.TraceID = sc.TraceID().String() // o mesmo ID do trace do
OpenTelemetry (Módulo 11)
    }
}
```

```

w.Header().Set("Content-Type", "application/problem+json")
w.WriteHeader(d.Status)
_ = json.MarshalWrite(w, d)
}

```

A opção `embed` do `encoding/json/v2` (chamada `inline` durante a fase experimental) achata o mapa `Extensoes` no objeto, então `arquivoExistenteId` aparece ao lado de `title` e `status`, como o padrão pede.

4.3 Erros como valores: o mapeamento central

Três tipos de erro cobrem quase tudo que uma API devolve:

```

package dominio

// Sentinela: compare com errors.Is
var ErrNaoEncontrado = errors.New("não encontrado")

// ErroDeNegocio: desfechos esperados com código estável e status HTTP
type ErroDeNegocio struct {
    Codigo    string
    Titulo    string
    Status    int
    Extensoes map[string]any
}

func (e *ErroDeNegocio) Error() string { return e.Codigo + ": " + e.Titulo }

func ArquivoDuplicado(existente uuid.UUID) error {
    return &ErroDeNegocio{
        Codigo:    "arquivo-duplicado",
        Titulo:    "Este arquivo já foi recebido.",
        Status:    http.StatusConflict,
        Extensoes: map[string]any{"arquivoExistenteId": existente},
    }
}

```

```

package validacao

// ErrosDeCampo (do Módulo 3) passa a implementar error
type ErrosDeCampo map[string][]string

func (e ErrosDeCampo) Add(campo, msg string) { e[campo] = append(e[campo], msg) }
func (e ErrosDeCampo) Error() string          { return "requisição inválida" }

func Campo(campo, msg string) error { return ErrosDeCampo{campo: {msg}} }

```

E uma única função decide o status de cada um. Ela usa `errors.AsType`, a versão genérica de `errors.As` que chegou no Go 1.26: mais segura e mais legível, sem a variável de destino declarada antes.

```

func DeErro(w http.ResponseWriter, r *http.Request, err error) {
    if e, ok := errors.AsType[*dominio.ErroDeNegocio](err); ok {
        Escrever(w, r, Detalhes{Type: BaseTipos + e.Codigo, Title: e.Titulo,
        Status: e.Status, Extensoes: e.Extensoes})
        return
    }
    if e, ok := errors.AsType[validacao.ErrosDeCampo](err); ok {
        Escrever(w, r, Detalhes{Type: BaseTipos + "validacao", Title:
        "Requisição inválida.", Status: http.StatusBadRequest, Errors: e})
        return
    }
    if _, ok := errors.AsType[*http.MaxBytesError](err); ok {
        Escrever(w, r, Detalhes{Title: "Corpo da requisição grande demais.",
        Status: http.StatusRequestEntityTooLarge})
        return
    }
    switch {
    case errors.Is(err, dominio.ErrNaoEncontrado):
        Escrever(w, r, Detalhes{Title: "Recurso não encontrado.", Status:
        http.StatusNotFound})
    case errors.Is(err, context.Canceled):
        // o cliente desistiu: não há a quem responder
    case errors.Is(err, context.DeadlineExceeded):
        Escrever(w, r, Detalhes{Title: "Tempo de processamento esgotado.",
        Status: http.StatusGatewayTimeout})
    default:
        slog.ErrorContext(r.Context(), "erro não tratado", "erro", err,
        "rota", r.Pattern)
    }
}

```

```

        Escrever(w, r, Detalhes{Title: "Erro interno.", Status:
http.StatusInternalServerError})
    }
}

```

O único lugar onde a mensagem original de um erro inesperado aparece é o log. A resposta 500 nunca a expõe, porque ela pode conter SQL, nomes de tabela ou caminhos internos. Para que `slog.ErrorContext` use o logger configurado, chame `slog.SetDefault(log)` no `run`.

Como os erros atravessam camadas embrulhados com `%w`, o repositório pode devolver `fmt.Errorf("obter arquivo %s: %w", id, dominio.ErrNaoEncontrado)` e o mapeamento continua reconhecendo o 404, enquanto o log ganha o contexto.

4.4 Validação: o tipo valida a si mesmo

Go não tem o `AddValidation` do .NET 10. O idioma é cada tipo de entrada ter um método `Validar`, e uma função genérica que lê e valida de uma vez:

```

type Validavel interface {
    Validar() validacao.ErrosDeCampo
}

func lerEValidar[T Validavel](w http.ResponseWriter, r *http.Request, limite
int64) (T, error) {
    v, err := lerJSON[T](w, r, limite)
    if err != nil {
        return v, err
    }
    if erros := v.Validar(); len(erros) > 0 {
        return v, erros
    }
    return v, nil
}

type RegistrarCallbackRequest struct {
    URL      string `json:"url"`
    Eventos []string `json:"eventos"`
}

func (p RegistrarCallbackRequest) Validar() validacao.ErrosDeCampo {
    e := validacao.ErrosDeCampo{}
    switch u, err := url.Parse(p.URL); {
    case p.URL == "":
        e.Add("url", "obrigatório")
    case len(p.URL) > 500:

```

```

    e.Add("url", "no máximo 500 caracteres")
    case err != nil || u.Scheme != "https" || u.Host == "":
        e.Add("url", "deve ser uma URL https absoluta")
    }
    if len(p.Eventos) == 0 {
        e.Add("eventos", "informe pelo menos um evento")
    }
    return e
}

```

E no handler, uma linha: `pedido, err := lerEValidar[RegistrarCallbackRequest](w, r, 16<<10); if err != nil { return err }`. A função `lerJSON` do Módulo 3 também passa a embrulhar erros de JSON malformado num `validacao.ErrosDeCampo{"corpo": {...}}`, deixando passar intacto só o `*http.MaxBytesError`, para que cada um vire o status certo.

Validação em código é mais verbosa que atributos, mas é Go puro: testável, sem reflexão e com regras entre campos (como `de` antes de `ate`) escritas do jeito mais natural. Se o time preferir declarativo, a biblioteca `go-playground/validator` usa tags como `validate:"required,url"`; o custo é uma mini-linguagem em strings que o compilador não verifica.

4.5 Validação não é regra de negócio

A mesma separação do curso de .NET vale aqui: `Validar()` verifica forma (400) e nunca acessa o banco; regras de estado ("este arquivo já foi recebido", "este título já foi liquidado") moram no serviço e voltam como `*ErroDeNegocio` com 409 ou 422. A verificação de duplicidade de verdade continua sendo o `UNIQUE` do banco, porque só ela é à prova de concorrência.

4.6 Recuperação de panic

Um `panic` num handler é um bug. O servidor do Go o recupera e fecha a conexão, mas o cliente não recebe nada útil e o log padrão não tem o trace da requisição. O middleware mais externo da cadeia resolve isso:

```

func ComRecuperacao(log *slog.Logger) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request)
        {
            defer func() {
                v := recover()
                if v == nil {
                    return
                }
                if v == http.ErrAbortHandler {
                    panic(v) // aborto intencional (usado por proxies): deixa
o servidor tratar
                }
                log.ErrorContext(r.Context(), "panic no handler",
                    "valor", v, "rota", r.Pattern, "pilha",
string(debug.Stack()))
                problema.Escrever(w, r, problema.Detalhes{Title: "Erro
interno.", Status: http.StatusInternalServerError})
            }()
            next.ServeHTTP(w, r)
        })
    }
}

```

O `recover` só funciona na mesma goroutine. Se um handler dispara uma goroutine e ela entra em `panic`, o processo inteiro cai. Por isso goroutines criadas dentro de handlers precisam do próprio `defer func() { if v := recover(); v != nil { ... } }()`, ou melhor ainda, devem ser evitadas (Módulo 9).

Exercício do Módulo 4

Converta os handlers do Módulo 3 para o tipo `Handler` que devolve erro. Implemente `problema.Escrever`, `problema.DeErro` e `ComRecuperacao`. Faça chamadas que produzam 400 (UUID inválido e corpo com campo desconhecido), 404, 409 (duplicidade simulada), 413 (corpo grande) e 500 (um `panic` proposital) e confirme que as seis respostas têm `Content-Type: application/problem+json` e o mesmo formato. Compare com as respostas da API em .NET: um cliente deve conseguir tratar as duas com o mesmo código.

Módulo 5 — Organização do código

Uma API em Go se mantém legível quando cada feature é um **pacote** com seus handlers, tipos e regras; quando as interfaces são declaradas no pacote que as **usa**, com só os métodos necessários; e quando o `main` (ou o `run`) monta todo o grafo de dependências à mão, de cima para baixo. Não há contêiner, não há varredura de assembly e não há magia: tudo que existe na API está escrito em algum lugar que você consegue abrir.

5.1 Layout do módulo

```

cobranca-api/
├─ go.mod
├─ cmd/
│   └─ api/
│       └─ main.go          main + run: configuração e composição
├─ api/
│   └─ openapi.yaml        o contrato (Módulo 7)
├─ internal/
│   └─ arquivos/          feature: upload, consulta, listagem,
progresso
│   │   └─ handlers.go
│   │   └─ contratos.go   structs de entrada e saída
│   │   └─ recebimento.go regra do upload
│   │   └─ consulta.go    leituras
│   └─ titulos/
│   └─ ocorrencias/
│   └─ dominio/          NossoNumero, StatusArquivo, ErroDeNegocio,
ErrNaoEncontrado
│   └─ banco/            código gerado pelo sqlc (Módulo 6) +
migrations
│   └─ plataforma/
│       └─ httpx/        Handler, Grupo, Encadear, JSON, lerJSON
│       └─ problema/    ProblemDetails
│       └─ validacao/    ErrosDeCampo
│       └─ auth/         JWT, usuário no contexto, escopos
└─ tests/               testes de integração com containers (Módulo
10)

```

Três escolhas desse layout:

- **internal/** é uma regra do compilador: pacotes dentro dele só podem ser importados pelo próprio módulo. Nenhum outro projeto passa a depender dos seus detalhes por acidente.

- **Pacote por feature**, não por camada. Não existe um pacote `handlers`, outro `services` e outro `models`; existe `arquivos`, com tudo de arquivos. Uma mudança de negócio toca uma pasta.
- **plataforma/** guarda o que é transversal e não sabe nada de cobrança. Se um pacote dali importar `arquivos`, algo está errado. Evite também o pacote `utils`: um nome que não diz o que o pacote faz vira depósito de tudo.

Um aviso que pega quem vem do .NET: Go **proíbe importações circulares**. Se `arquivos` importa `titulos` e `titulos` importa `arquivos`, não compila. Isso força uma direção clara: features dependem de `dominio` e `plataforma`, nunca umas das outras. O que duas features compartilham sobe para `dominio`.

5.2 Uma feature: interfaces no consumidor

O pacote `arquivos` declara as interfaces de que precisa, pequenas e não exportadas. Quem as implementa nem sabe que elas existem:

```
package arquivos

// Só o que estes handlers usam. A implementação real (com pgx) e a falsa
(nos testes) satisfazem isto.
type consulta interface {
    Obter(ctx context.Context, clienteID, id uuid.UUID) (ArquivoResposta,
error)
    Listar(ctx context.Context, clienteID uuid.UUID, f Filtro, antesDe int64)
(Pagina[ArquivoResumo], error)
    Pertence(ctx context.Context, clienteID, id uuid.UUID) (bool, error)
}

type recebimento interface {
    Receber(ctx context.Context, p PedidoDeRecebimento, corpo io.Reader)
(ArquivoResposta, error)
}

type Handlers struct {
    consulta      consulta
    recebimento   recebimento
}

func NovosHandlers(c consulta, r recebimento) *Handlers {
    return &Handlers{consulta: c, recebimento: r}
}

// Registrar é o MapGrupoArquivos do curso de .NET: o índice das rotas da
feature
```

```

func (h *Handlers) Registrar(v1 httpx.Grupo) {
    g := v1.Sub("/arquivos")
    g.Rota("POST", "", httpx.Handler(h.receber),
auth.ExigirEscopo("arquivos:escrever"))
    g.Rota("GET", "", httpx.Handler(h.listar))
    g.Rota("GET",("/{id}", httpx.Handler(h.obter), h.exigirDono)
    g.Rota("GET",("/{id}/progresso", httpx.Handler(h.progresso),
h.exigirDono)
}

func (h *Handlers) obter(w http.ResponseWriter, r *http.Request) error {
    id, err := uuid.Parse(r.PathValue("id"))
    if err != nil {
        return validacao.Campo("id", "deve ser um UUID")
    }
    arquivo, err := h.consulta.Obter(r.Context(),
auth.Usuario(r.Context()).ClienteID, id)
    if err != nil {
        return err
    }
    return httpx.JSON(w, http.StatusOK, arquivo)
}

// Pagina é a resposta de listagem com cursor (Módulo 6)
type Pagina[T any] struct {
    Itens []T `json:"itens"`
    ProximoCursor string `json:"proximoCursor,omitzero"`
}

```

Compare com o .NET: lá o handler recebia `IConsultaDeArquivos` do contêiner, e a interface morava junto da implementação. Em Go, a interface mora junto de quem a consome, e por isso ela nunca tem métodos que o consumidor não usa. No teste do handler, a implementação falsa é uma struct de cinco linhas.

5.3 Middleware de feature: o dono do arquivo

O equivalente ao filtro `ExigirDonoDoArquivo` do .NET é um middleware de rota. Ele roda depois que o mux casou o padrão, então `r.PathValue("id")` já está preenchido, e reaproveita o tipo `httpx.Handler` para devolver erro em vez de escrever a resposta:

```

func (h *Handlers) exigirDono(next http.Handler) http.Handler {
    return httpx.Handler(func(w http.ResponseWriter, r *http.Request) error {
        id, err := uuid.Parse(r.PathValue("id"))
        if err != nil {
            return validacao.Campo("id", "deve ser um UUID")
        }
        pertence, err := h.consulta.Pertence(r.Context(),
auth.Usuario(r.Context()).ClienteID, id)
        if err != nil {
            return err
        }
        if !pertence {
            return dominio.ErrNaoEncontrado // 404, nunca 403: não confirme
que o ID existe
        }
        next.ServeHTTP(w, r)
        return nil
    })
}

```

Como no curso de .NET, ele é uma defesa a mais, não a única: `consulta.Obter` também filtra pelo cliente. As duas juntas impedem que uma rota nova esquecida sem o middleware vazze dados.

5.4 O run como raiz de composição

Todo o grafo de dependências é montado em um lugar, na ordem em que as coisas precisam existir:

```

func run(ctx context.Context, getenv func(string) string, saida io.Writer)
error {
    cfg, err := carregarConfig(getenv)
    if err != nil {
        return fmt.Errorf("configuração: %w", err)
    }
    log := slog.New(slog.NewJSONHandler(saida, nil))
    slog.SetDefault(log)

    pool, err := pgxpool.New(ctx, cfg.BancoURL)
    if err != nil {
        return fmt.Errorf("banco: %w", err)
    }
    defer pool.Close()

    validador, err := auth.NovoValidador(ctx, cfg.JWTEmissor,

```

```

cfg.JWTAudencia)
    if err != nil {
        return fmt.Errorf("jwt: %w", err)
    }

consultas := banco.New(pool) // código gerado pelo sqlc
storage := armazenamento.NovoS3(cfg.Bucket)

arquivosH := arquivos.NovosHandlers(
    arquivos.NovaConsulta(consultas),
    arquivos.NovoRecebimento(pool, storage),
)
titulosH := titulos.NovosHandlers(titulos.NovaConsulta(consultas))

mux := http.NewServeMux()
v1 := httpx.NovoGrupo(mux, "/v1", auth.Autenticar(validador),
limites.PorCliente(100))
arquivosH.Registrar(v1)
titulosH.Registrar(v1)
mux.Handle("GET /health/live", saude.Vivo())
mux.Handle("GET /health/ready", saude.NovaProntidao(pool))

raiz := httpx.Encadear(mux, httpx.ComRecuperacao(log),
otelhttp.NewMiddleware("cobranca-api"), httpx.ComLog(log))
return servir(ctx, cfg, raiz) // Módulo 11: timeouts + desligamento
gracioso
}

```

O que esse estilo dá em troca da verbosidade:

- **O compilador verifica o grafo.** Uma dependência faltando não compila. Não existe o erro de "serviço não registrado" em tempo de execução.
- **Tempos de vida são visíveis.** O `pool` vive o processo inteiro porque é criado no `run` e fechado no `defer`. Não há escopo por requisição; o que é da requisição está no `context.Context`.
- **Health checks fora do grupo `/v1`** não recebem autenticação nem rate limiting, sem precisar de um `AllowAnonymous`: eles simplesmente não estão no grupo que aplica esses middlewares.

Quando o `run` passar de umas cem linhas, extraia funções por área (`montarBanco`, `montarRotas`), mas mantenha a composição explícita. Geradores de código de injeção de dependência existem em Go, mas raramente compensam numa API deste tamanho.

5.5 Quanta arquitetura em camadas?

Os mesmos critérios do curso de .NET, com uma vantagem em Go: o `sqlc` (Módulo 6) gera funções tipadas a partir do SQL, então a camada de consultas já nasce fina e testável.

- **Leituras:** o serviço de consulta chama as funções geradas e converte o resultado para o tipo de resposta. Nada de repositório genérico.
- **Escritas com regra:** um serviço com a regra de negócio e a transação (`recebimento.go`), testável sem HTTP.
- **Domínio compartilhado** em `internal/dominio`. Se no futuro outro serviço Go precisar dele, mova-o para um módulo próprio; os workers em .NET do primeiro curso compartilham o banco, não o código.

O Go 1.27 trouxe **métodos genéricos**: um método agora pode declarar os próprios parâmetros de tipo. Funções auxiliares como `lerJSON[T]` podem passar a ser métodos de um tipo de plataforma, em vez de funções soltas do pacote, se isso deixar a API do pacote mais clara. Interfaces continuam sem métodos genéricos.

5.6 O que evitar

- **Variáveis globais com dependências** (`var db *pgxpool.Pool` no pacote): escondem o grafo e tornam os testes dependentes de ordem.
- **`init()` com efeito colateral** (abrir conexões, ler arquivos): roda em toda importação, inclusive nos testes, antes de qualquer configuração.
- **Interfaces grandes declaradas "para o caso de precisar"**, junto da implementação: o idioma é aceitar interfaces e devolver structs concretas.
- **Um pacote `models` com todas as structs**: vira o centro de todas as importações e acaba causando ciclos.

Exercício do Módulo 5

Organize o que você construiu até aqui no layout de 5.1, com os pacotes `arquivos` e `titulos`, cada um com `NovosHandlers` e `Registrar`. Implemente `exigirDono` e aplique-o a `GET /v1/arquivos/{id}`. Escreva um teste do handler `obter` com uma `consulta` falsa (uma struct com os três métodos) e confira que o arquivo de outro cliente devolve 404. Por fim, tente fazer `titulos` importar `arquivos` e `arquivos` importar `titulos` e leia o erro de ciclo do compilador.

Módulo 6 — Dados: pgx, sqlc, paginação, streaming e upload

Em Go, o caminho mais produtivo para PostgreSQL é escrever SQL de verdade e deixar uma ferramenta gerar o código Go tipado a partir dele: o **sqlc**, sobre o driver **pgx**. Não há ORM com change tracker; há consultas que você lê e funções geradas que o compilador verifica. As regras de desempenho são as mesmas do .NET: só as colunas necessárias, keyset em vez de **OFFSET**, streaming em vez de materializar, upload sem passar pela memória e **POST** idempotente.

6.1 pgx e o pool de conexões

O pgx é o driver PostgreSQL de referência em Go. Ele pode ser usado pela interface genérica `database/sql`, mas a interface nativa é mais rápida e dá acesso a recursos do PostgreSQL (`COPY`, `LISTEN/NOTIFY`, tipos como arrays e `jsonb`). Para uma API, use o pool nativo:

```
cfgPool, err := pgxpool.ParseConfig(cfg.BancoURL)
if err != nil {
    return fmt.Errorf("banco: %w", err)
}
cfgPool.MaxConns = 20 // teto de conexões desta
                        réplica
cfgPool.MinConns = 2 // conexões quentes para a
                      primeira requisição
cfgPool.MaxConnLifetime = 30 * time.Minute // recicla conexões (failover,
balanceamento)
cfgPool.HealthCheckPeriod = 30 * time.Second

pool, err := pgxpool.NewWithConfig(ctx, cfgPool)
```

`MaxConns` é o equivalente ao `Maximum Pool Size` do `Npgsql`, e a mesma conta do curso de arquivos vale aqui: `réplicas × MaxConns` precisa caber no orçamento de conexões que o banco reserva para a API. O pool é seguro para uso concorrente; crie um por processo e passe-o adiante.

6.2 sqlc: SQL primeiro, Go gerado

Você escreve as consultas num arquivo `.sql`, cada uma com um nome e uma cardinalidade:

```

-- name: ObterArquivo :one
SELECT id, nome_original, banco, status, recebido_em,
       linhas_processadas, linhas_rejeitadas, motivo
FROM arquivo
WHERE cliente_id = @cliente_id AND id = @id;

-- name: ListarArquivos :many
SELECT id, nome_original, status, recebido_em, sequencia
FROM arquivo
WHERE cliente_id = @cliente_id
  AND sequencia < @antes_de
  AND (sqlc.narg('status')::text IS NULL OR status = sqlc.narg('status'))
  AND (sqlc.narg('de')::date IS NULL OR data_recebimento >=
sqlc.narg('de'))
  AND (sqlc.narg('ate')::date IS NULL OR data_recebimento <=
sqlc.narg('ate'))
ORDER BY sequencia DESC
LIMIT @limite;

```

O `sqlc generate` lê essas consultas junto com as migrations (o schema da tabela `arquivo` do curso de arquivos, Módulo 2.3), valida cada uma contra o schema e gera funções como `ObterArquivo(ctx, ObterArquivoParams) (ObterArquivoRow, error)`. Uma coluna renomeada no banco quebra o `sqlc generate`, não a produção. `sqlc.narg` declara um parâmetro anulável, que é como filtros opcionais viram uma consulta só.

Duas configurações do `sqlc.yaml` que valem a pena: `emit_pointers_for_null_types: true` (colunas anuláveis viram `*string` em vez de `pgtype.Text`) e um `override` de tipo para que colunas `uuid` usem o `uuid.UUID` da biblioteca padrão. Como ele é um `[16]byte`, confirme que a sua versão do `pgx` o escaneia diretamente; se não, converta a partir de `pgtype.UUID` na borda do pacote de consulta.

O serviço de consulta converte o resultado gerado para o tipo de resposta e traduz "nenhuma linha" para o erro de domínio:

```

func (c *Consulta) Obter(ctx context.Context, clienteID, id uuid.UUID)
(ArquivoResposta, error) {
    l, err := c.q.ObterArquivo(ctx, banco.ObterArquivoParams{ClienteID:
clienteID, ID: id})
    if errors.Is(err, pgx.ErrNoRows) {
        return ArquivoResposta{}, dominio.ErrNaoEncontrado
    }
    if err != nil {
        return ArquivoResposta{}, fmt.Errorf("obter arquivo %s: %w", id, err)
    }
    return ArquivoResposta{
        ID: l.ID, NomeOriginal: l.NomeOriginal, Banco: l.Banco,
        Status: dominio.StatusArquivo(l.Status), RecebidoEm: l.RecebidoEm,
        LinhasProcessadas: int(l.LinhasProcessadas), LinhasRejeitadas:
int(l.LinhasRejeitadas),
        Motivo: valorOuVazio(l.Motivo), // *string (coluna anulável) para
string
    }, nil
}

```

6.3 Paginação keyset com cursor opaco

O mesmo desenho do curso de .NET (Módulo 6.3): a coluna `sequencia`, o índice `(cliente_id, sequencia DESC)`, pedir `limite + 1` linhas para saber se há próxima página e um cursor opaco em Base64 URL-safe:

```

func codificarCursor(sequencia int64) string {
    return
base64.RawURLEncoding.EncodeToString(binary.BigEndian.AppendUint64(nil,
uint64(sequencia)))
}

func decodificarCursor(s string) (int64, error) {
    if s == "" {
        return math.MaxInt64, nil // sem cursor: começa do mais recente
    }
    b, err := base64.RawURLEncoding.DecodeString(s)
    if err != nil || len(b) != 8 {
        return 0, validacao.Campo("cursor", "cursor inválido")
    }
    return int64(binary.BigEndian.Uint64(b)), nil
}

func (h *Handlers) listar(w http.ResponseWriter, r *http.Request) error {
    filtro, erros := lerFiltro(r.URL.Query())

```

```

    if filtro.Limite < 1 || filtro.Limite > 200 {
        erros.Add("limite", "deve estar entre 1 e 200")
    }
    if len(erros) > 0 {
        return erros
    }
    antesDe, err := decodificarCursor(filtro.Cursor)
    if err != nil {
        return err
    }
    pagina, err := h.consulta.Listar(r.Context(),
auth.Usuario(r.Context()).ClienteID, filtro, antesDe)
    if err != nil {
        return err
    }
    return httpx.JSON(w, http.StatusOK, pagina)
}

```

No serviço, `Listar` chama `ListarArquivos` com `Limite: filtro.Limite + 1`, corta o excedente e, se havia excedente, gera o `ProximoCursor` a partir da `sequencia` do último item. A sequência fica fora do JSON (`json: "-"` na struct de resumo): ela existe só para montar o cursor.

6.4 Respostas em streaming

O `IAsyncEnumerable<T>` do .NET não tem equivalente automático, mas o resultado é simples de obter: percorrer as linhas do pgx e escrever cada uma direto no `ResponseWriter`. Para a exportação em CSV:

```

func (h *Handlers) exportar(w http.ResponseWriter, r *http.Request) error {
    dia, err := time.Parse(time.DateOnly, r.URL.Query().Get("dia"))
    if err != nil {
        return validacao.Campo("dia", "use o formato AAAA-MM-DD")
    }
    rc := http.NewResponseController(w)
    _ = rc.SetWriteDeadline(time.Now().Add(10 * time.Minute)) // esta rota
    pode demorar

    linhas, err := h.pool.Query(r.Context(),
        `SELECT nosso_numero, codigo_ocorrencencia, data_ocorrencencia,
valor_pago_centavos
FROM retorno_detalhe d JOIN arquivo a ON a.id = d.arquivo_id
WHERE a.cliente_id = $1 AND d.data_ocorrencencia = $2`,
auth.Usuario(r.Context()).ClienteID, dia)
    if err != nil {

```

```

        return err // ainda não escrevemos nada: vira ProblemDetails
normalmente
    }
    defer linhas.Close()

    w.Header().Set("Content-Type", "text/csv; charset=utf-8")
    w.Header().Set("Content-Disposition", `attachment; filename="ocorrencias-
`+dia.Format(time.DateOnly)+`.csv"`)
    csvW := csv.NewWriter(w)
    csvW.Comma = ';'
    _ = csvW.Write([]string{"nosso_numero", "ocorrencia", "data",
"valor_centavos"})

    var (
        nosso, valor int64
        codigo        int16
        data          time.Time
        n             int
    )
    for linhas.Next() {
        if err := linhas.Scan(&nosso, &codigo, &data, &valor); err != nil {
            break
        }
        _ = csvW.Write([]string{strconv.FormatInt(nosso, 10),
strconv.Itoa(int(codigo)), data.Format(time.DateOnly),
strconv.FormatInt(valor, 10)})
        if n++; n%5000 == 0 {
            csvW.Flush() // esvazia o buffer do csv no ResponseWriter...
            _ = rc.Flush() // ...e o ResponseWriter na rede
        }
    }
    csvW.Flush()
    if err := cmp.Or(linhas.Err(), csvW.Error()); err != nil {
        slog.ErrorContext(r.Context(), "exportação interrompida", "erro",
err)
        panic(http.ErrAbortHandler) // corta a conexão: o cliente percebe que
o arquivo veio incompleto
    }
    return nil
}

```

O detalhe mais importante está no fim. Depois que o primeiro byte saiu, o status 200 já foi enviado e um erro não vira mais ProblemDetails. Se a leitura falhar no meio e o handler simplesmente retornar, o cliente recebe um CSV truncado que parece completo.

`panic(http.ErrAbortHandler)` é a forma prevista pela biblioteca padrão de abortar a

resposta: o servidor fecha a conexão sem terminar o corpo, o cliente vê erro de transferência, e o middleware de recuperação do Módulo 4 deixa esse valor passar de propósito.

Se preferir isolar a leitura do banco da escrita HTTP, os iteradores do Go (`iter.Seq2[T, error]`, desde o Go 1.23) permitem que o serviço devolva uma sequência e o handler faça `for o, err := range h.servico.Ocorrencias(ctx, dia)`, com o mesmo streaming por baixo.

6.5 Upload de arquivos grandes

Como no .NET, o caminho é o corpo cru: `r.Body` já é um `io.Reader` em streaming, e o arquivo nunca precisa existir inteiro na memória ou no disco local.

```
func (h *Handlers) receber(w http.ResponseWriter, r *http.Request) error {
    if mt, _, _ := mime.ParseMediaType(r.Header.Get("Content-Type")); mt !=
"application/octet-stream" {
        return &dominio.ErroDeNegocio{Codigo: "tipo-de-conteudo", Status:
http.StatusUnsupportedMediaType,
            Titulo: "Envie o arquivo como application/octet-stream."}
    }
    q := r.URL.Query()
    pedido := PedidoDeRecebimento{
        ClienteID:      auth.Usuario(r.Context()).ClienteID,
        NomeArquivo:     q.Get("nomeArquivo"),
        Banco:          q.Get("banco"),
        ChaveIdempotencia: r.Header.Get("Idempotency-Key"),
    }
    if erros := pedido.Validar(); len(erros) > 0 {
        return erros
    }

    _ = http.NewResponseController(w).SetReadDeadline(time.Now().Add(15 *
time.Minute)) // só nesta rota
    corpo := http.MaxBytesReader(w, r.Body, 500<<20)
    // 500 MiB

    arquivo, err := h.recebimento.Receber(r.Context(), pedido, corpo)
    if err != nil {
        return err
    }
    w.Header().Set("Location", "/v1/arquivos/"+arquivo.ID.String())
    return httpx.JSON(w, http.StatusAccepted, arquivo)
}
```

E o serviço, que repete o fluxo de ingestão do curso de arquivos (hash calculado enquanto envia, deduplicação pelo `UNIQUE (cliente_id, sha256)`, outbox na mesma transação):

```
func (s *Recebimento) Receber(ctx context.Context, p PedidoDeRecebimento,
corpo io.Reader) (ArquivoResposta, error) {
    id := uuid.NewV7() // ordenado por tempo: inserções no fim do índice
    chave := "retornos/" + time.Now().UTC().Format("2006/01/02") + "/" +
id.String()

    hash := sha256.New()
    if err := s.storage.Enviar(ctx, chave, io.TeeReader(corpo, hash)); err !=
nil {
        return ArquivoResposta{}, fmt.Errorf("enviar ao storage: %w", err)
    }
    sha := hex.EncodeToString(hash.Sum(nil)) // completo: o envio leu o corpo
até o fim

    var resp ArquivoResposta
    err := pgx.BeginFunc(ctx, s.pool, func(tx pgx.Tx) error {
        q := s.q.WithTx(tx)
        criado, err := q.InserirArquivo(ctx, banco.InserirArquivoParams{
            ID: id, ClienteID: p.ClienteID, Banco: p.Banco, NomeOriginal:
p.NomeArquivo,
            Sha256: sha, ChaveStorage: chave, DataRecebimento:
hojeEmSaoPaulo(),
        }) // INSERT ... ON CONFLICT (cliente_id, sha256) DO NOTHING
RETURNING ...
        if errors.Is(err, pgx.ErrNoRows) {
            existente, err := q.ArquivoPorHash(ctx,
banco.ArquivoPorHashParams{ClienteID: p.ClienteID, Sha256: sha})
            if err != nil {
                return err
            }
            return dominio.ArquivoDuplicado(existente)
        }
        if err != nil {
            return err
        }
        if err := q.InserirOutbox(ctx, eventoArquivoRecebido(criado)); err !=
nil {
            return err
        }
        resp = paraResposta(criado)
        return nil
    })
    if err != nil {
```

```

    _ = s.storage.Remove(context.WithoutCancel(ctx), chave) // duplicado
    ou falha: descarta o objeto
    return ArquivoResposta{}, err
}
return resp, nil
}

```

Peças que merecem atenção:

- **io.TeeReader** passa cada byte lido pelo storage também pelo hash: uma leitura, dois destinos. O **Enviar** usa o upload multipart do SDK da AWS (**feature/s3/manager**), que aceita um **io.Reader** de tamanho desconhecido e envia em partes.
- **pgx.BeginFunc** abre a transação, faz commit se a função devolve **nil** e rollback se devolve erro (ou se entra em pânico). É o padrão para não esquecer o rollback.
- **context.WithoutCancel(ctx)** na limpeza: se o erro foi o cliente desistir, o contexto da requisição já está cancelado e a remoção falharia com ele. A limpeza precisa acontecer mesmo assim.
- **O 413**: quando o corpo passa de 500 MiB, o **MaxBytesReader** devolve um ***http.MaxBytesError** dentro da leitura do SDK. Confira num teste se o erro que chega ao handler ainda o carrega (via **%w**); se o SDK não o preservar, verifique o tamanho lido no próprio serviço.

6.6 Idempotência no POST

O algoritmo é o do curso de .NET (Módulo 6.6, com o diagrama de decisão): **INSERT** da chave com **ON CONFLICT DO NOTHING** , executar só se a chave for nova, devolver a resposta gravada se ela já terminou, 409 se está em andamento e 422 se a chave foi reutilizada para outra operação. Em Go, ele cabe num middleware que grava a resposta do handler embrulhando o **ResponseWriter** :

```

type gravadorDeResposta struct {
    http.ResponseWriter
    status int
    corpo  bytes.Buffer
}

func (g *gravadorDeResposta) WriteHeader(s int) { g.status = s;
g.ResponseWriter.WriteHeader(s) }
func (g *gravadorDeResposta) Write(b []byte) (int, error) {
    if g.status == 0 {
        g.status = http.StatusOK
    }
    g.corpo.Write(b) // cópia para gravar; só serve para respostas pequenas,
como o 202 do upload
    return g.ResponseWriter.Write(b)
}
func (g *gravadorDeResposta) Unwrap() http.ResponseWriter { return
g.ResponseWriter }

```

O middleware faz o `INSERT` da chave antes de chamar o handler, e depois grava `status`, os headers relevantes (`Location`) e `corpo` na mesma linha. Numa repetição concluída, ele escreve a resposta gravada e nem chama o handler. Aplique-o só às rotas que aceitam `Idempotency-Key`.

6.7 Transações e concorrência otimista

Para atualizações concorrentes, a coluna de sistema `xmin` do PostgreSQL serve de versão, como no curso de .NET. Com SQL explícito, a verificação fica visível na própria consulta:

```

-- name: AtualizarCallback :execrows
UPDATE callback SET url = @url, eventos = @eventos
WHERE id = @id AND cliente_id = @cliente_id AND xmin::text = @versao;

```

No `GET`, devolva `xmin` como `ETag`; no `PUT`, exija `If-Match` e passe o valor como `versao`. Se `:execrows` devolver zero linhas afetadas, ou o recurso não existe, ou outra requisição o alterou antes: uma consulta de existência decide entre 404 e 412 `Precondition Failed`.

Exercício do Módulo 6

Configure o `sqlc` com as migrations da tabela `arquivo` e gere as consultas de 6.2. Implemente `GET /v1/arquivos` com cursor e confira com `EXPLAIN` que a página 100 custa o mesmo que a primeira. Implemente o upload e envie 400 MB com `curl --data-`

`binary`, acompanhando a memória do processo pelo endpoint `/debug/pprof/heap` (Módulo 9): ela não deve crescer com o tamanho do arquivo. Por fim, interrompa uma exportação no meio (derrube o banco) e confirme que o cliente recebe erro de transferência, não um CSV truncado com status 200.

Módulo 7 — Contrato: OpenAPI como fonte da verdade

No ASP.NET Core o documento OpenAPI é gerado a partir do código, porque o framework conhece os tipos de cada parâmetro e de cada `TypedResults`. Em Go, um handler `func(w, r)` não diz nada sobre o que lê ou devolve, então não há de onde gerar. A prática dominante inverte o fluxo: **o contrato é escrito primeiro** em `openapi.yaml`, e o `oapi-codegen` gera dele os tipos Go e uma interface de servidor que o seu código implementa. O compilador passa a garantir que o código cumpre o contrato.

7.1 As três abordagens em Go

Abordagem	Como funciona	Prós	Contras
Spec-first com oapi-codegen	<code>openapi.yaml</code> → tipos e interface de servidor gerados	O contrato é revisado como documento; o código não diverge dele	Escrever YAML; regenerar a cada mudança
Code-first com Huma	Tipos Go de entrada e saída declaram o contrato; Huma gera o OpenAPI	Experiência próxima do .NET	Handlers na forma do Huma, não <code>func(w, r)</code>
Comentários com swagger	Anotações em comentários viram OpenAPI	Funciona em código existente	Comentários não são verificados pelo compilador e divergem com o tempo

Este curso usa a primeira: é a mais comum em Go para APIs de integração e dá a garantia mais forte, porque o documento revisado no pull request é exatamente o que o código implementa.

7.2 O contrato

Um trecho do `api/openapi.yaml` para a consulta de arquivo:

```

openapi: 3.1.0
info:
  title: API de Cobrança
  version: v1
security:
  - bearer: []
paths:
  /v1/arquivos/{id}:
    get:
      operationId: ObterArquivo
      tags: [Arquivos]
      summary: Situação de um arquivo de retorno
      parameters:
        - name: id
          in: path
          required: true
          schema: { type: string, format: uuid }
      responses:
        "200":
          description: Arquivo encontrado
          content:
            application/json:
              schema: { $ref: "#/components/schemas/ArquivoResposta" }
        "404":
          $ref: "#/components/responses/NaoEncontrado"
components:
  securitySchemes:
    bearer: { type: http, scheme: bearer, bearerFormat: JWT }
  schemas:
    ArquivoResposta:
      type: object
      required: [id, nomeOriginal, banco, status, recebidoEm,
linhasProcessadas, linhasRejeitadas]
      properties:
        id: { type: string, format: uuid }
        nomeOriginal: { type: string }
        banco: { type: string }
        status: { type: string, enum: [RECEBIDO, PROCESSANDO, CONCLUIDO,
REJEITADO, FALHOU] }
        recebidoEm: { type: string, format: date-time }
        linhasProcessadas: { type: integer }
        linhasRejeitadas: { type: integer }
        motivo: { type: string }
    ProblemDetails:
      type: object
      required: [type, title, status]

```

```

properties:
  type: { type: string }
  title: { type: string }
  status: { type: integer }
  detail: { type: string }
  instance: { type: string }
  traceId: { type: string }
  errors:
    type: object
    additionalProperties: { type: array, items: { type: string } }
responses:
  NaoEncontrado:
    description: Recurso inexistente ou de outro cliente
    content:
      application/problem+json:
        schema: { $ref: "#/components/schemas/ProblemDetails" }

```

Repare que o `security` no topo resolve o que no .NET exigia um transformer: todas as operações exigem bearer, e a exceção (se houver) declara `security: []` na própria operação. O schema de `ProblemDetails` é compartilhado por todas as respostas de erro.

7.3 Gerando o servidor

A configuração do `oapi-codegen` pede tipos, um servidor para o `net/http` padrão e o modo **strict**:

```

# api/oapi-codegen.yaml
package: api
output: internal/api/gerado.go
generate:
  models: true
  std-http-server: true    # usa o ServeMux do Go 1.22+
  strict-server: true      # handlers recebem e devolvem objetos tipados

```

Com `go tool oapi-codegen -config api/oapi-codegen.yaml api/openapi.yaml` (registre o gerador como ferramenta do módulo com `go get -tool`), o pacote `api` passa a ter os tipos do contrato e uma interface com um método por `operationId`. No modo **strict**, cada método recebe um objeto de requisição já convertido e devolve uma das respostas declaradas:

```
func (s *Servidor) ObterArquivo(ctx context.Context, req
api.ObterArquivoRequestObject) (api.ObterArquivoResponseObject, error) {
    a, err := s.consulta.Obter(ctx, auth.Usuario(ctx).ClienteID, req.Id)
    if errors.Is(err, dominio.ErrNaoEncontrado) {
        return api.ObterArquivo404ApplicationProblemPlusJSONResponse{
            NaoEncontradoApplicationProblemPlusJSONResponse:
problemaNaoEncontrado(ctx),
        }, nil
    }
    if err != nil {
        return nil, err // cai no tratador de erros configurado no servidor
strict
    }
    return api.ObterArquivo200JSONResponse(paraContrato(a)), nil
}
```

Esse é o equivalente direto de `Results<Ok<ArquivoResposta>, NotFound>`: o tipo de retorno só aceita as respostas que o contrato declara. Adicionar um 409 exige mudá-lo no YAML, regenerar, e só então o código compila com a nova resposta. Os nomes exatos dos tipos gerados seguem o `operationId`, o status e o content type, e mudam se você mudar esses três; abra o arquivo gerado na primeira vez para conhecê-los.

Três ajustes de integração:

- **UUID:** por padrão o gerador usa um tipo de UUID de biblioteca externa. A extensão `x-go-type` no schema permite apontar para o `uuid.UUID` da biblioteca padrão, mantendo um tipo só no projeto todo.
- **Erros:** o servidor strict aceita funções de tratamento para erros de decodificação da requisição e para erros devolvidos pelos métodos. Ligue as duas ao `problema.DeErro` do Módulo 4, para que tudo continue saindo em `ProblemDetails`.
- **Middlewares:** o servidor gerado é um `http.Handler` comum, então entra na cadeia do Módulo 2 normalmente. Middlewares de rota podem ser aplicados por operação.

7.4 Onde o modo strict não serve

O modo strict brilha em endpoints JSON comuns e atrapalha nos três casos especiais da API de cobrança: o upload em corpo cru (500 MB não devem passar por um objeto de requisição), a exportação em CSV (streaming) e o progresso em SSE (resposta que não termina). A solução pragmática: essas três operações continuam descritas no `openapi.yaml` (o contrato é único), mas são implementadas como handlers comuns do Módulo 6, registrados no mesmo mux. Um teste de contrato (7.5) garante que elas respeitam o documento.

7.5 O contrato no CI

O documento já está no repositório desde o primeiro dia, e o CI faz três coisas com ele:

1. **Regenera e compara:** roda o `oapi-codegen` e falha se o código gerado commitado estiver desatualizado. Ninguém muda o YAML sem regenerar.
2. **Valida o estilo:** um linter de OpenAPI (Spectral ou Redocly) verifica regras do time, como `operationId` em toda operação e `ProblemDetails` em todo erro.
3. **Detecta quebras:** `oasdiff` compara o YAML do pull request com o da branch principal e falha em mudanças incompatíveis (campo removido, tipo alterado, parâmetro que virou obrigatório).

E, nos testes de integração, um middleware de validação contra o documento (o projeto `oapi-codegen` mantém um para `net/http`, baseado no `kin-openapi`) confere requisições e respostas reais. Em produção, use-o só para requisições, se usar: validar respostas tem custo e não é o cliente quem deve pagar por um bug seu.

7.6 Publicando a documentação

O arquivo pode ser embutido no binário com `//go:embed` e servido junto com uma página estática do Scalar (ou do Redoc), sem dependência de runtime:

```
//go:embed openapi.yaml
var especificacao []byte

mux.HandleFunc("GET /openapi.yaml", func(w http.ResponseWriter, r
*http.Request) {
    w.Header().Set("Content-Type", "application/yaml")
    _, _ = w.Write(especificacao)
})
```

Como no .NET, decida conscientemente se esse endpoint fica público ou só dentro da rede.

7.7 Versionamento

As regras do curso de .NET (Módulos 7.7 e 7.8) valem integralmente: versão no caminho da URL, nova versão só para mudanças incompatíveis, cuidado com valores novos em enums, `Deprecation` e `Sunset` para aposentar. Em Go, cada versão é um documento (`api/v1/openapi.yaml`, `api/v2/openapi.yaml`) e um pacote gerado (`internal/api/v1`, `internal/api/v2`). A implementação da v2 pode reaproveitar os mesmos serviços de domínio e só converter para os tipos novos na borda.

Exercício do Módulo 7

Escreva o `openapi.yaml` com as operações `ObterArquivo` e `ListarArquivos`, gere o servidor strict e implemente as duas. Ligue o tratador de erros do servidor strict ao `problema.DeErro`. Depois adicione um status 410 que o código devolve mas o YAML não declara e veja o compilador recusar. Por fim, configure o `oasdiff` no CI e faça uma mudança incompatível de propósito (remover `banco` de `ArquivoResposta`) para ver o build falhar.

Módulo 8 — Segurança: quem pode chamar o quê, e quantas vezes

As camadas são as mesmas do curso de .NET: autenticação por JWT emitido por um provedor externo, autorização por escopo, autorização por recurso e limites por cliente. Em Go, cada camada é um middleware de poucas linhas que você escreve e lê, e "seguro por padrão" vem da estrutura: todas as rotas de negócio estão num grupo que aplica autenticação, e uma rota fora desse grupo é visível no `run`.

8.1 Validando o JWT

A API não emite tokens, só os valida: assinatura (com as chaves públicas do provedor, publicadas como JWKS), emissor, audiência e validade. A biblioteca `github.com/coreos/go-oidc/v3` faz a descoberta (`/.well-known/openid-configuration`), baixa e guarda em cache as chaves e verifica tudo isso:

```
package auth

type Identidade struct {
    Sujeito    string
    ClienteID  uuid.UUID
    Escopos    []string
}

type Validador struct {
    verificador *oidc.IDTokenVerifier
}

// ctx deve ser o contexto do processo (vive enquanto a API vive), não o de
// uma requisição
func NovoValidador(ctx context.Context, emissor, audiencia string)
(*Validador, error) {
    provedor, err := oidc.NewProvider(ctx, emissor) // descoberta + JWKS com
    cache e rotação de chaves
    if err != nil {
        return nil, fmt.Errorf("descoberta OIDC em %s: %w", emissor, err)
    }
}
```

```

    }
    return &Validador{verificador: provedor.Verifier(&oidc.Config{ClientID:
audiencia})}, nil
}

func (v *Validador) Validar(ctx context.Context, token string) (Identidade,
error) {
    t, err := v.verificador.Verify(ctx, token) // assinatura, iss, aud (o
ClientID acima) e exp
    if err != nil {
        return Identidade{}, err
    }
    var claims struct {
        ClienteID string `json:"cliente_id"`
        Scope      string `json:"scope"`
    }
    if err := t.Claims(&claims); err != nil {
        return Identidade{}, err
    }
    id, err := uuid.Parse(claims.ClienteID)
    if err != nil {
        return Identidade{}, errors.New("token sem cliente_id válido")
    }
    return Identidade{Sujeito: t.Subject, ClienteID: id, Escopos:
strings.Fields(claims.Scope)}, nil
}

```

O nome `IDTokenVerifier` engana um pouco: ele verifica qualquer JWT assinado pelo provedor, inclusive os tokens de acesso do fluxo *client credentials* que os parceiros usam, desde que o provedor os emita como JWT com a audiência da API. O campo `ClientID` da configuração é a audiência esperada; nunca deixe-o vazio nem ligue `SkipClientIDCheck` em produção, ou tokens emitidos para outras APIs da empresa serão aceitos aqui. A claim `cliente_id` é a mesma do curso de .NET (Módulo 8.1): configure-a no provedor.

`strings.Fields` resolve a armadilha dos escopos: a claim `scope` vem com valores separados por espaço, e comparar a string inteira falharia sempre que o token tivesse mais de um escopo.

8.2 Os middlewares de autenticação e de escopo

```

// Verificador: o que Autenticar precisa. *Validador satisfaz; os testes usam
um falso (Módulo 10)
type Verificador interface {
    Validar(ctx context.Context, token string) (Identidade, error)
}

```

```

func Autenticar(v Verificador) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request)
        {
            token, ok := strings.CutPrefix(r.Header.Get("Authorization"),
"Bearer ")
            if !ok || token == "" {
                naoAutenticado(w, r)
                return
            }
            id, err := v.Validar(r.Context(), token)
            if err != nil {
                naoAutenticado(w, r) // o motivo vai para o log, nunca para a
resposta
                slog.InfoContext(r.Context(), "token recusado", "erro", err)
                return
            }
            next.ServeHTTP(w, r.WithContext(ComIdentidade(r.Context(), id)))
        })
    }
}

func naoAutenticado(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("WWW-Authenticate", `Bearer realm="cobranca"`)
    problema.Escrever(w, r, problema.Detalhes{Title: "Não autenticado.",
Status: http.StatusUnauthorized})
}

func ExigirEscopo(escopo string) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request)
        {
            if !slices.Contains(Usuario(r.Context()).Escopos, escopo) {
                problema.Escrever(w, r, problema.Detalhes{Title: "Escopo
insuficiente.", Status: http.StatusForbidden})
                return
            }
            next.ServeHTTP(w, r)
        })
    }
}

type chaveIdentidade struct{}

// ComIdentidade coloca a identidade no contexto (usada pelo middleware e

```

```

pelos testes)
func ComIdentidade(ctx context.Context, id Identidade) context.Context {
    return ctx.WithValue(ctx, chaveIdentidade{}, id)
}

// Usuario devolve a identidade colocada por Autenticar. Chamar numa rota sem
Autenticar é bug: entra em pânico.
func Usuario(ctx context.Context) Identidade {
    id, ok := ctx.Value(chaveIdentidade{}).(Identidade)
    if !ok {
        panic("auth.Usuario chamado numa rota sem o middleware Autenticar")
    }
    return id
}

```

É a versão final do exemplo de contexto do Módulo 2.6. O `panic` em `Usuario` é proposital: uma rota de negócio registrada fora do grupo autenticado não deve funcionar "com cliente vazio" (o que, com filtros por cliente, devolveria listas vazias em vez de um erro visível). Ela deve quebrar alto no primeiro teste, com uma mensagem que diz o que faltou, e o middleware de recuperação transforma isso num 500 registrado.

8.3 Autorização por recurso

A primeira falha da lista OWASP API Security Top 10, *Broken Object Level Authorization*, é a mesma em qualquer linguagem. As defesas também:

- **Middleware na rota** (`exigirDono` , Módulo 5.3).
- **Filtro em toda consulta**: toda consulta do sqlc de dados de cliente tem `WHERE cliente_id = @cliente_id`, e as funções geradas exigem o parâmetro, então esquecê-lo não compila.
- **Row Level Security no PostgreSQL** (opcional, a defesa mais forte): políticas na tabela só expõem linhas do cliente configurado na sessão. Em cada transação, a API executa `SELECT set_config('app.cliente_id', $1, true)`, e a política `USING (cliente_id = current_setting('app.cliente_id')::uuid)` faz o resto. Mesmo uma consulta escrita errado não enxerga dados de outro cliente. O custo é configurar a variável em toda transação e um usuário de banco sem o atributo `BYPASSRLS`.

E, como sempre: objeto de outro cliente responde 404.

8.4 Rate limiting por cliente

O pacote `golang.org/x/time/rate` implementa um token bucket. Um limitador por cliente, guardado num mapa protegido por mutex:

```

package limites

type porCliente struct {
    mu      sync.Mutex
    limites map[uuid.UUID]*entrada
    taxa    rate.Limit
    rajada  int
    limpeza time.Time
}

type entrada struct {
    lim    *rate.Limiter
    visto time.Time
}

func PorCliente(porMinuto int) func(http.Handler) http.Handler {
    p := &porCliente{
        limites: map[uuid.UUID]*entrada{},
        taxa:    rate.Every(time.Minute / time.Duration(porMinuto)), // uma
        ficha a cada 600 ms para 100/min
        rajada:  porMinuto, // até
        100 de uma vez
    }
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request)
        {
            res := p.obter(auth.Usuario(r.Context()).ClienteID).Reserve()
            if espera := res.Delay(); espera > 0 {
                res.Cancel() // devolve a reserva: esta requisição será
                recusada, não adiada
                w.Header().Set("Retry-After",
                strconv.Itoa(int(math.Ceil(espera.Seconds()))))
                problema.Escrever(w, r, problema.Detalhes{Title: "Limite de
                requisições excedido.", Status: http.StatusTooManyRequests})
                return
            }
            next.ServeHTTP(w, r)
        })
    }
}

func (p *porCliente) obter(cliente uuid.UUID) *rate.Limiter {
    p.mu.Lock()
    defer p.mu.Unlock()
    agora := time.Now()
    if agora.Sub(p.limpezaEm) > time.Minute { // limpeza oportunista:

```

```

clientes parados há 10 min saem do mapa
    for k, e := range p.limites {
        if agora.Sub(e.vistoEm) > 10*time.Minute {
            delete(p.limites, k)
        }
    }
    p.limpezaEm = agora
}
e, ok := p.limites[cliente]
if !ok {
    e = &entrada{lim: rate.NewLimiter(p.taxa, p.rajada)}
    p.limites[cliente] = e
}
e.vistoEm = agora
return e.lim
}

```

Três observações de projeto:

- **Reposição contínua.** O `rate.Limiter` repõe fichas continuamente (uma a cada 600 ms), enquanto a configuração do curso de .NET repunha 100 de uma vez a cada minuto. A taxa sustentada é a mesma; o comportamento após uma rajada é mais suave, e o `Retry-After` costuma ser de um segundo, não de até um minuto.
- **Ordem no grupo.** O limitador lê o `ClienteID`, então precisa vir depois de `Autenticar` na lista de middlewares do grupo `/v1` (Módulo 5.4).
- **Por réplica.** Exatamente como no .NET, o mapa vive na memória de cada processo: com 4 réplicas, o limite efetivo é cerca de 4 vezes o configurado. Para limites contratuais exatos, aplique-os no gateway ou num armazenamento compartilhado (Redis).

Para limitar uploads simultâneos por cliente, o padrão de Go é um semáforo, que pode ser um canal com buffer por cliente (`make(chan struct{}, 2)`): o middleware tenta enviar no canal sem bloquear (`select` com `default`), recusa com 429 se estiver cheio e libera com `<-sem` num `defer` ao fim da requisição.

8.5 CORS e proteção entre origens

CORS só importa se um portal web chama a API do navegador. Nesse caso, a biblioteca `github.com/rs/cors` é um middleware `http.Handler` que se configura com a lista explícita de origens, métodos e headers, as mesmas do curso de .NET (incluindo `expor Location` e `Retry-After`). Coloque-o na cadeia global, antes da autenticação, porque as requisições *preflight* (`OPTIONS`) não trazem token.

O Go 1.25 adicionou `http.CrossOriginProtection`, que recusa requisições não seguras vindas de outra origem no navegador, usando os headers `Sec-Fetch-Site` e `Origin`. Ele protege contra CSRF, um risco de APIs autenticadas por **cookie**. Com tokens bearer, que o navegador não anexa sozinho, ele não é necessário; se um dia a API aceitar cookies, ligue-o.

8.6 O resto da lista

- **TLS** no ingress; o tráfego interno do cluster conforme a política da empresa.
- **Limites de entrada:** `MaxBytesReader` por rota, `MaxHeaderBytes` e os timeouts do `http.Server` (Módulo 2.1).
- **Segredos** vêm de variáveis de ambiente injetadas pelo cofre ou pelos Secrets do Kubernetes, lidas uma vez no `carregarConfig`.
- **Logs sem dados sensíveis:** nunca registre o header `Authorization`, e mascare dados pessoais de pagadores (LGPD). Um `slog.Handler` com `ReplaceAttr` pode mascarar atributos sensíveis de forma central.
- **Vulnerabilidades:** rode `govulncheck ./...` no CI. Ao contrário de um scanner que só olha versões, ele analisa se o seu código de fato chama a função vulnerável, o que reduz muito o ruído.

Exercício do Módulo 8

Suba um Keycloak em Docker com dois clientes de máquina e escopos diferentes, como no curso de .NET. Implemente `Autenticar`, `ExigirEscopo` e `PorCliente` e teste: sem token (401 com `WWW-Authenticate`), token com audiência errada (401), escopo insuficiente (403), arquivo de outro cliente (404) e 101 chamadas rápidas (a última com 429 e `Retry-After: 1`). Depois registre uma rota de negócio fora do grupo `/v1` e confirme que `auth.Usuario` quebra com a mensagem do `panic`, devolvendo um 500 registrado.

Módulo 9 — Concorrência e performance sob carga

Um servidor Go já é concorrente por construção: uma goroutine por requisição, escalonadas pelo runtime sobre os núcleos disponíveis. O trabalho de performance numa API, então, não é "paralelizar", e sim não desperdiçar: não repetir consultas (cache com proteção contra stampede), não esperar além do necessário (contextos com prazo), não vaziar goroutines e configurar o runtime para o container. E, como sempre, medir antes, aqui com o `pprof` embutido.

9.1 Medir: pprof e o perfil de goroutines vazadas

O pacote `net/http/pprof` expõe perfis de CPU, heap, goroutines e bloqueio num endpoint HTTP. Ele **nunca** deve ficar na porta pública: sirva-o num segundo servidor, numa porta só acessível de dentro do cluster:

```
admin := http.NewServeMux()
admin.HandleFunc("/debug/pprof/", pprof.Index)
admin.HandleFunc("/debug/pprof/profile", pprof.Profile)
admin.HandleFunc("/debug/pprof/trace", pprof.Trace)
go func() { _ = (&http.Server{Addr: "127.0.0.1:6060", Handler:
admin}).ListenAndServe() }()
```

Com ele no ar, `go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30` captura 30 segundos de CPU sob carga e abre o flame graph (a visão padrão desde o Go 1.26). `.../heap` mostra quem está alocando memória; `.../goroutine` mostra o que cada goroutine está fazendo.

O Go 1.27 tornou geral o perfil `goroutineleak` (`/debug/pprof/goroutineleak`): ele usa o próprio GC para encontrar goroutines bloqueadas em canais ou mutexes que nenhuma outra goroutine consegue mais desbloquear. Num servidor HTTP, vazamento de goroutine é o equivalente a vazamento de memória lento, e esse perfil o encontra sem adivinhação.

9.2 O runtime no container

- **GOMAXPROCS**: desde o Go 1.25 o runtime respeita o limite de CPU do cgroup do container. Um pod com limite de 2 CPUs num nó de 64 núcleos usa 2, sem configuração.
- **GOMEMLIMIT**: o GC do Go, por padrão, só olha o crescimento do heap (`GOGC`), não o limite do container. Defina `GOMEMLIMIT` um pouco abaixo do limite de memória do pod (por exemplo, cerca de 90%): o GC fica mais agressivo quando se aproxima dele, em vez de o processo ser morto por OOM. É o equivalente ao `GCHepHardLimitPercent` do .NET, com a diferença de ser um limite "suave".
- **GC**: o Green Tea, coletor padrão desde o Go 1.26, reduziu bastante o custo de marcação de objetos pequenos, e o Go 1.27 especializou a alocação de objetos pequenos por tamanho. Você ganha isso só por compilar com a versão nova; não há o que configurar.

9.3 Goroutines dentro de handlers

A regra: **nenhuma goroutine criada num handler pode viver mais que a requisição**, a menos que alguém seja dono dela de forma explícita. Fire-and-forget (`go enviarEmail()`) perde erros, não é esperado no desligamento e, se entrar em pânico, derruba o processo inteiro (Módulo 4.6). Trabalho que precisa acontecer depois da resposta vai para a outbox (Módulo 6.5), não para uma goroutine.

Quando um handler precisa de duas consultas independentes ao mesmo tempo, use `errgroup`, que cancela as irmãs se uma falhar e espera todas antes de seguir:

```
g, ctx := errgroup.WithContext(r.Context())
var (
    titulo dominio.Titulo
    total  int
)
g.Go(func() (err error) { titulo, err = h.titulos.Obter(ctx, cliente,
numero); return })
g.Go(func() (err error) { total, err = h.ocorrencias.Contar(ctx, cliente,
numero); return })
if err := g.Wait(); err != nil {
    return err
}
```

`g.SetLimit(n)` limita quantas rodam ao mesmo tempo, o que importa quando o número de tarefas vem da entrada. Para o caso simples sem erro, `sync.WaitGroup.Go` (Go 1.25) dispensa o `Add / Done` manual. E lembre que cada consulta concorrente é uma conexão a mais do pool: paralelizar dentro da requisição multiplica o uso de conexões sob carga.

9.4 Prazos: do servidor até o banco

Os timeouts do `http.Server` (Módulo 2.1) protegem a conexão, mas não limitam quanto tempo o seu código leva. Para isso, um middleware coloca um prazo no contexto, e o `pgx`, o cliente HTTP e tudo que recebe o contexto passam a respeitá-lo:

```
func ComPrazo(d time.Duration) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request)
        {
            ctx, cancel := context.WithTimeout(r.Context(), d)
            defer cancel()
            next.ServeHTTP(w, r.WithContext(ctx))
        })
    }
}
```

Quando o prazo estoura, a consulta no banco é cancelada, o repositório devolve um erro que embrulha `context.DeadlineExceeded` e o `problema.DeErro` responde 504 (Módulo 4.3). Aplique 10 segundos rota a rota, nas rotas comuns de cada feature, e não no grupo inteiro: um contexto só encurta prazos, nunca os alonga, e upload, exportação e SSE precisam de mais. Ou seja, prazo curto nas rotas comuns do grupo `/v1` e prazos próprios

nas rotas de exportação e upload. Evite `http.TimeoutHandler` nesta API: ele guarda a resposta inteira num buffer para poder substituí-la por um erro, o que quebra streaming e SSE.

Nas chamadas de saída, nunca use `http.DefaultClient` (ele não tem timeout). Crie um `http.Client` com `Timeout` e um `Transport` ajustado, uma vez, e compartilhe-o. Retry com backoff e jitter e circuit breaker não vêm na biblioteca padrão; os princípios são os do curso de .NET (Módulo 9.5), e bibliotecas como `sony/gobreaker` oferecem o circuit breaker.

9.5 Cache com proteção contra stampede

O `HybridCache` do .NET combina cache local, cache distribuído e proteção contra stampede. Em Go, a peça central da proteção é o `singleflight`: várias goroutines pedindo a mesma chave ao mesmo tempo esperam uma única execução.

```
type CacheDeTitulos struct {
    grupo singleflight.Group
    mu     sync.RWMutex
    itens map[string]itemCache
    ttl   time.Duration
    fonte func(ctx context.Context, cliente uuid.UUID, n dominio.NossoNumero)
        (dominio.Titulo, error)
}

type itemCache struct {
    titulo dominio.Titulo
    err     error // ErrNaoEncontrado também fica em cache
    expira  time.Time
}

func (c *CacheDeTitulos) Obter(ctx context.Context, cliente uuid.UUID, n
dominio.NossoNumero) (dominio.Titulo, error) {
    chave := cliente.String() + ":" + strconv.FormatInt(int64(n), 10) // o
    cliente faz parte da chave

    c.mu.RLock()
    it, ok := c.itens[chave]
    c.mu.RUnlock()
    if ok && time.Now().Before(it.expira) {
        return it.titulo, it.err
    }

    v, err, _ := c.grupo.Do(chave, func() (any, error) {
        // contexto próprio: se o primeiro chamador desistir, os que esperam
```

```

não falham junto
    ctxBusca, cancel := context.WithTimeout(context.WithoutCancel(ctx),
2*time.Second)
    defer cancel()
    t, err := c.fonte(ctxBusca, cliente, n)
    if err != nil && !errors.Is(err, dominio.ErrNaoEncontrado) {
        return dominio.Titulo{}, err // falha técnica: não vai para o
cache
    }
    c.mu.Lock()
    c.itens[chave] = itemCache{titulo: t, err: err, expira:
time.Now().Add(c.ttl)}
    c.mu.Unlock()
    return t, err
})
return v.(dominio.Titulo), err
}

```

O detalhe sutil é o `context.WithoutCancel`: a função do `singleflight` roda com o contexto do primeiro chamador, e se ele desconectar, todos os outros que estavam esperando receberiam "contexto cancelado" sem culpa. A busca ganha um contexto próprio, desligado do cancelamento mas com prazo.

O que esse cache não faz: não limita o tamanho (use uma biblioteca de LRU ou TTL se houver muitas chaves) e não é compartilhado entre réplicas. Para um nível distribuído, coloque um Redis atrás da `fonte`. A invalidação quando o pipeline atualiza um título segue o curso de .NET: por evento da outbox, não só por expiração.

9.6 Server-Sent Events: progresso em tempo real

SSE em Go é escrever no `ResponseWriter` no formato do protocolo e dar `Flush` a cada evento:

```

func (h *Handlers) progresso(w http.ResponseWriter, r *http.Request) error {
    id, err := uuid.Parse(r.PathValue("id"))
    if err != nil {
        return validacao.Campo("id", "deve ser um UUID")
    }
    rc := http.NewResponseController(w)
    _ = rc.SetWriteDeadline(time.Time{}) // sem prazo de escrita nesta rota

    w.Header().Set("Content-Type", "text/event-stream")
    w.Header().Set("Cache-Control", "no-cache")
    w.Header().Set("X-Accel-Buffering", "no") // pede ao NGINX para não fazer
buffering
    w.WriteHeader(http.StatusOK)
}

```

```

// a partir daqui, o handler só devolve nil: a resposta já começou

ctx, cliente := r.Context(), auth.Usuario(r.Context()).ClienteID
tick := time.NewTicker(time.Second)
defer tick.Stop()

var anterior Progresso
for i := 0; ; i++ {
    atual, err := h.consulta.Progresso(ctx, cliente, id)
    if err != nil {
        return nil // encerra o stream; o EventSource do navegador
reconecta sozinho
    }
    switch {
    case atual != anterior: // Progresso é uma struct comparável
        dados, _ := json.Marshal(atual)
        fmt.Fprintf(w, "event: progresso\ndata: %s\n\n", dados)
        anterior = atual
    case i%15 == 0:
        fmt.Fprintf(w, ": ping\n\n") // comentário SSE: mantém a conexão
viva em proxies
    }
    if rc.Flush() != nil || atual.Final() {
        return nil
    }
    select {
    case <-ctx.Done(): // o cliente fechou
        return nil
    case <-h.desligando: // o servidor está desligando (Módulo 11)
        return nil
    case <-tick.C:
    }
}
}
}

```

Pontos que só aparecem em produção:

- **rc.Flush()** chega ao writer verdadeiro mesmo através dos middlewares que o embrulham, graças ao **Unwrap** do Módulo 2.3. Sem ele, os eventos ficariam presos num buffer até o fim da conexão.
- **Comentários : ping** a cada 15 segundos sem mudança impedem que o ingress feche a conexão por inatividade.
- **Depois do WriteHeader , só return nil** . Devolver erro faria o adaptador do Módulo 4 tentar escrever um ProblemDetails no meio do stream.

- **Desligamento:** `srv.Shutdown` não cancela o contexto de requisições em andamento. Sem o ajuste do Módulo 11 (`h.desligando`), um stream aberto seguraria o desligamento até o tempo-limite.

Como no curso de .NET, consultar o banco a cada segundo por conexão serve para dezenas de conexões; para milhares, troque por notificações. O `pgx` suporta `LISTEN/NOTIFY` nativamente: o worker faz `NOTIFY` progresso, `'<id>'`, e uma goroutine única da API escuta e distribui para os streams interessados.

9.7 Compressão

Se o ingress já comprime, não faça nada. Se não, um middleware como `gzhttp` (do pacote `klauspost/compress`) comprime respostas acima de um tamanho mínimo, respeita `Accept-Encoding` e lida corretamente com `Flush` (necessário para SSE). As mesmas ressalvas de BREACH do curso de .NET se aplicam.

9.8 As causas mais comuns de lentidão em Go

- **Esquecer o contexto** numa chamada de I/O: trabalho continua depois que o cliente desistiu, e o prazo do Módulo 9.4 não tem efeito.
- **Goroutines sem dono** ou bloqueadas para sempre em canais: crescem até esgotar a memória. O perfil `goroutineleak` existe para isso.
- **Segurar um mutex durante I/O**: todas as requisições que precisam do mutex esperam a rede. Copie o que precisa, solte o lock, depois faça I/O.
- **Não fechar `resp.Body`** de chamadas HTTP de saída: a conexão não volta ao pool. No Go 1.27, o `Close` de respostas HTTP/1 também esvazia automaticamente o que não foi lido (até um limite), o que melhora o reaproveitamento, mas o `defer resp.Body.Close()` continua obrigatório.
- **N+1 consultas e alocação em laços quentes** (`fmt.Sprintf` por item): o flame graph do `pprof` mostra ambos em minutos.

Exercício do Módulo 9

Com o servidor de `pprof` na porta de administração, rode um teste de carga de 2 minutos em `GET /v1/titulos/{nossoNumero}` e capture um perfil de CPU. Adicione o `CacheDeTitulos` e compare o p95. Depois escreva de propósito um handler que cria uma goroutine que espera num canal que ninguém fecha, chame-o algumas vezes e encontre o vazamento em `/debug/pprof/goroutineleak`. Por fim, abra o stream de progresso no navegador enquanto o worker processa um arquivo, e confirme que os eventos chegam um a um, não todos no fim.

Módulo 10 — Testes: provando que tudo funciona

Go traz quase tudo para testar uma API na biblioteca padrão: o pacote `testing` (com testes de tabela, subtestes, benchmarks e fuzzing), `net/http/httpptest` (requisições e servidores em memória) e, desde o Go 1.25, `testing/synctest` para código que depende do relógio. Para o banco real, `testcontainers-go`. A estratégia é a mesma do curso de .NET: a maior parte do valor está nos testes de integração que atravessam a cadeia inteira de middlewares contra um PostgreSQL de verdade.

10.1 O que testar em cada nível

Nível	O que cobre	Ferramenta	Equivalente no .NET
Unidade	Tipos de domínio, <code>Validar()</code> , regras	<code>testing</code> com tabelas	xUnit puro
Handler	Um handler com dependências falsas	<code>httpptest.NewRecorder</code>	Teste do método estático com <code>TypedResults</code>
Integração	Cadeia completa + banco real	<code>httpptest.NewServer</code> + <code>testcontainers-go</code>	<code>WebApplicationFactory</code> + <code>Testcontainers</code>
Tempo	Rate limiting, cache, prazos	<code>testing/synctest</code>	<code>FakeTimeProvider</code>
Propriedades	Parsers com entradas inesperadas	Fuzzing nativo (<code>go test -fuzz</code>)	Bibliotecas de property testing

10.2 Testes de tabela

O idioma de Go para testar muitos casos de uma função é uma tabela de casos percorrida com subtestes:

```
func TestParseNossoNumero(t *testing.T) {
    casos := []struct {
        nome      string
        entrada   string
        quer      dominio.NossoNumero
        querErro  bool
    }{
```

```

{"simples", "12345", 12345, false},
{"zeros à esquerda", "00012345", 12345, false},
{"vazio", "", 0, true},
{"letras", "12a45", 0, true},
{"zero", "000", 0, true},
{"longo demais", strings.Repeat("9", 19), 0, true},
}
for _, c := range casos {
    t.Run(c.nome, func(t *testing.T) {
        t.Parallel()
        got, err := dominio.ParseNossoNumero(c.entrada)
        if (err != nil) != c.querErro {
            t.Fatalf("erro = %v; queria erro = %v", err, c.querErro)
        }
        if got != c.quer {
            t.Errorf("= %d; queria %d", got, c.quer)
        }
    })
}
}

```

Cada linha da tabela vira um subteste com nome (`go test -run 'TestParseNossoNumero/letras'` roda só um), e `t.Parallel()` os roda em paralelo. Desde o Go 1.22, a variável do `for` é nova a cada iteração, então capturar `c` dentro do subteste paralelo é seguro.

10.3 Testando um handler isolado

Como os handlers dependem de interfaces pequenas (Módulo 5.2), o falso é uma struct, e o `httptest.NewRecorder` captura a resposta:

```

type consultaFalsa struct{ arquivo *arquivos.ArquivoResposta }

func (c consultaFalsa) Obter(context.Context, uuid.UUID, uuid.UUID)
(arquivos.ArquivoResposta, error) {
    if c.arquivo == nil {
        return arquivos.ArquivoResposta{}, dominio.ErrNaoEncontrado
    }
    return *c.arquivo, nil
}
// ... Listar e Pertence do mesmo jeito

func TestObterArquivoInexistenteDevolve404(t *testing.T) {
    mux := http.NewServeMux()
    arquivos.NovosHandlers(consultaFalsa{},
nil).Registrar(httpx.NovoGrupo(mux, "/v1"))
}

```

```

    req := httptest.NewRequest(http.MethodGet,
"/v1/arquivos/"+uuid.NewV7().String(), nil)
    req = req.WithContext(auth.ComIdentidade(req.Context(),
auth.Identidade{ClienteID: uuid.NewV7()}))
    rec := httptest.NewRecorder()

    mux.ServeHTTP(rec, req)

    if rec.Code != http.StatusNotFound {
        t.Fatalf("status = %d; queria 404", rec.Code)
    }
    if ct := rec.Header().Get("Content-Type"); ct !=
"application/problem+json" {
        t.Errorf("Content-Type = %q", ct)
    }
}

```

Passar pelo `mux` em vez de chamar o método direto faz o teste exercitar também o padrão de rota e o `PathValue`. A função `auth.ComIdentidade` é exportada justamente para testes e para o middleware: ela coloca a identidade no contexto com a chave privada do pacote.

10.4 Integração: o equivalente ao `WebApplicationFactory`

O segredo é separar a montagem do handler do `run`. Uma função `MontarHandler` recebe as dependências já prontas e devolve o `http.Handler` completo (mux, grupos, middlewares); o `run` a chama com as dependências reais, e o teste com as de teste. O que muda entre produção e teste fica explícito numa struct:

```

type Dependencias struct {
    Pool      *pgxpool.Pool
    Storage   armazenamento.Storage
    Verificador auth.Verificador // a interface do Módulo 8, exportada para
    cá
    Log       *slog.Logger
}

```

O banco sobe uma vez por pacote de teste, no `TestMain`:

```

var urlBanco string

func TestMain(m *testing.M) {
    ctx := context.Background()
    pg, err := postgres.Run(ctx, "postgres:17-alpine",
        postgres.WithDatabase("cobranca"),
        postgres.BasicWaitStrategies())
    if err != nil {
        log.Fatal(err)
    }
    urlBanco, _ = pg.ConnectionString(ctx, "sslmode=disable")
    if err := migracoes.Aplicar(ctx, urlBanco); err != nil {
        log.Fatal(err)
    }
    codigo := m.Run()
    _ = testcontainers.TerminateContainer(pg)
    os.Exit(codigo)
}

```

E cada teste monta um servidor HTTP real em memória, com um verificador de token falso que mapeia tokens fixos para identidades:

```

type verificadorFalso map[string]auth.Identidade

func (v verificadorFalso) Validar(_ context.Context, token string)
(auth.Identidade, error) {
    id, ok := v[token]
    if !ok {
        return auth.Identidade{}, errors.New("token desconhecido")
    }
    return id, nil
}

var (
    clienteA = auth.Identidade{ClienteID: uuid.NewV7(), Escopos:
[]string{"arquivos:escrever", "cobranca:ler"}}
    clienteB = auth.Identidade{ClienteID: uuid.NewV7(), Escopos:
[]string{"cobranca:ler"}}
)

func novoServidor(t *testing.T) *httptest.Server {
    t.Helper()
    pool, err := pgxpool.New(t.Context(), urlBanco)
    if err != nil {
        t.Fatal(err)
    }
}

```

```

    t.Cleanup(pool.Close)

    h := app.MontarHandler(app.Dependencias{
        Pool:      pool,
        Storage:    armazenamento.NovoEmMemoria(),
        Verificador: verificadorFalso{"token-a": clienteA, "token-b":
clienteB},
        Log:      slog.New(slog.DiscardHandler),
    })
    srv := httptest.NewServer(h)
    t.Cleanup(srv.Close)
    return srv
}

func TestClienteNaoEnxergaArquivoDeOutro(t *testing.T) {
    srv := novoServidor(t)

    envio := requisitar(t, srv, "token-a", http.MethodPost,
        "/v1/arquivos?nomeArquivo=ret.txt&banco=341",
bytes.NewReader(arquivoValido))
    if envio.StatusCode != http.StatusAccepted {
        t.Fatalf("upload: status %d; queria 202", envio.StatusCode)
    }
    local := envio.Header.Get("Location")

    if r := requisitar(t, srv, "token-a", http.MethodGet, local, nil);
r.StatusCode != http.StatusOK {
        t.Errorf("dono: status %d; queria 200", r.StatusCode)
    }
    if r := requisitar(t, srv, "token-b", http.MethodGet, local, nil);
r.StatusCode != http.StatusNotFound {
        t.Errorf("outro cliente: status %d; queria 404", r.StatusCode)
    }
}
}

```

`requisitar` é um helper de poucas linhas que monta a requisição, coloca `Authorization: Bearer <token>` e `Content-Type: application/octet-stream` quando há corpo, executa com `srv.Client()` e registra `t.Cleanup` para fechar o corpo da resposta. `t.Context()` (Go 1.24) é cancelado no fim do teste, e `slog.DiscardHandler` silencia os logs.

Com o verificador falso, o teste exercita tudo de verdade (middlewares, mux, escopos, banco, filtros por cliente) menos a criptografia do JWT, que é responsabilidade da biblioteca e tem testes próprios num nível abaixo. Os dados ficam isolados porque cada teste usa clientes próprios, como no curso de .NET.

10.5 Testando o tempo com syncctest

O rate limiter repõe uma ficha a cada 600 ms. Testar isso com `time.Sleep` real deixaria a suíte lenta e instável. O `testing/syncctest` roda o teste numa "bolha" com relógio falso: `time.Sleep` avança o tempo na hora, e todo código dentro da bolha (inclusive o `rate.Limiter`) enxerga esse tempo.

```
func TestLimiteDevolve429ERepoe(t *testing.T) {
    syncctest.Test(t, func(t *testing.T) {
        h := limites.PorCliente(100)
        (http.HandlerFunc(func(http.ResponseWriter, *http.Request) {}))
        id := auth.Identidade{ClienteID: uuid.NewV7()}
        chamar := func() *httptest.ResponseRecorder {
            req := httptest.NewRequest(http.MethodGet, "/", nil)
            rec := httptest.NewRecorder()
            h.ServeHTTP(rec,
                req.WithContext(auth.ComIdentidade(req.Context(), id)))
            return rec
        }

        for range 100 { // a rajada inteira passa
            if rec := chamar(); rec.Code != http.StatusOK {
                t.Fatalf("status %d dentro da rajada", rec.Code)
            }
        }
        rec := chamar()
        if rec.Code != http.StatusTooManyRequests || rec.Header().Get("Retry-After") != "1" {
            t.Fatalf("esperava 429 com Retry-After 1; veio %d %q", rec.Code,
                rec.Header().Get("Retry-After"))
        }

        time.Sleep(600 * time.Millisecond) // tempo falso: não espera de
        verdade
        if rec := chamar(); rec.Code != http.StatusOK {
            t.Fatalf("deveria ter repostado uma ficha; veio %d", rec.Code)
        }
    })
}
```

O mesmo vale para a expiração do cache e das chaves de idempotência. Para testes que precisam de um servidor HTTP inteiro dentro da bolha, o Go 1.27 adicionou `httptest.NewTestServer`, que usa uma rede falsa em memória compatível com o `syncctest`.

10.6 Fuzzing de parsers

Tudo que converte texto vindo de fora merece fuzzing. O motor nativo gera milhões de entradas a partir de exemplos e procura as que quebram uma propriedade:

```
func FuzzParseNossoNumero(f *testing.F) {
    for _, s := range []string{"1", "000123", "abc", "",
        "99999999999999999999"} {
        f.Add(s)
    }
    f.Fuzz(func(t *testing.T, s string) {
        n, err := dominio.ParseNossoNumero(s)
        if err != nil {
            return
        }
        if n <= 0 {
            t.Fatalf("aceitou %q como %d", s, n)
        }
        texto, _ := n.MarshalText()
        if string(texto) != strings.TrimLeft(s, "0") {
            t.Fatalf("ida e volta de %q deu %q", s, texto)
        }
    })
}
```

`go test -fuzz FuzzParseNossoNumero -fuzztime 30s` roda o fuzzer; entradas que falham são gravadas em `testdata/fuzz` e passam a rodar como testes normais para sempre.

10.7 Contrato e CI

Nos testes de integração, envolva o handler com o middleware de validação contra o `openapi.yaml` (Módulo 7.5), configurado para validar também as respostas: qualquer divergência entre o que a API devolve e o que o contrato promete vira falha de teste. No CI, rode sempre `go vet ./...` (o Go 1.27 inclui por padrão a verificação de uso de APIs mais novas que a versão declarada no `go.mod`), `go test -race ./...` e `govulncheck ./...`.

Exercício do Módulo 10

Monte o `TestMain` com o PostgreSQL em container, extraia `MontarHandler` do `run` e transforme em testes de integração os casos do exercício do Módulo 8 (401, 403, 404 de outro cliente), mais validação com 400 e `errors`, paginação sem repetição e

idempotência. Escreva o teste de `synctest` do rate limiter e um fuzz para `decodificarCursor`. Rode tudo com `-race` e meça o tempo total; deve ficar abaixo de um minuto.

Módulo 11 — Operação: desligamento, telemetria e deploy

Em produção a API precisa desligar sem derrubar requisições, dizer ao Kubernetes quando está pronta, emitir logs, métricas e traces correlacionados pelo mesmo `trace_id` e rodar numa imagem pequena e sem shell. Go facilita a última parte (um binário estático) e pede um pouco de código explícito nas outras três.

11.1 Desligamento gracioso

O `srv.ListenAndServe()` do Módulo 3 não sabe parar. O desligamento correto tem três passos, nesta ordem: avisar que não está mais pronto, esperar o balanceador parar de mandar tráfego e só então parar de aceitar conexões, esperando as requisições em andamento terminarem.

```
func servir(ctx context.Context, srv *http.Server, prontidao
*saude.Prontidao) error {
    erros := make(chan error, 1)
    go func() { erros <- srv.ListenAndServe() }()

    select {
    case err := <-erros:
        return fmt.Errorf("servidor: %w", err) // não subiu (porta ocupada,
por exemplo)
    case <-ctx.Done(): // SIGTERM ou Ctrl+C (signal.NotifyContext no main)
        slog.Info("desligando", "causa", context.Cause(ctx)) // desde o Go
1.26, a causa diz qual sinal chegou
    }

    prontidao.Desligar() // 1. /health/ready passa a responder 503
    time.Sleep(5 * time.Second) // 2. o balanceador tira a réplica de rota

    ctxDesl, cancel := context.WithTimeout(context.Background(),
25*time.Second)
    defer cancel()
    if err := srv.Shutdown(ctxDesl); err != nil { // 3. para de aceitar e
espera as ativas
        return fmt.Errorf("desligamento: %w", err)
    }
    return nil
}
```

E na montagem do servidor, um canal que avisa os streams longos:

```
desligando := make(chan struct{})
srv.RegisterOnShutdown(func() { close(desligando) }) // chamado quando o
Shutdown começa
```

O `srv.Shutdown` espera cada conexão ficar ociosa, mas **não cancela o contexto** das requisições em andamento. Um stream SSE nunca fica ocioso, então seguraria o desligamento até o tempo-limite. Por isso o handler de progresso do Módulo 9 também escuta `h.desligando`: ele encerra o stream, o `EventSource` do navegador reconecta e cai noutra réplica. Requisições comuns não são interrompidas: terminam normalmente dentro dos 25 segundos.

O `time.Sleep` do passo 2 faz no próprio processo o que o curso de .NET fez com `preStop`. Em Go isso é conveniente porque a imagem não tem shell (11.5). Configure o `terminationGracePeriodSeconds` do pod acima da soma (5 + 25 segundos, com folga: 40).

11.2 Health checks

```
package saude

type Prontidao struct {
    pool      *pgxpool.Pool
    desligando atomic.Bool
}

func (p *Prontidao) Desligar() { p.desligando.Store(true) }

// Vivo: só "o processo responde". Nunca depende do banco.
func Vivo() http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusOK) })
}

// Pronto: dependências essenciais + não estar desligando
func (p *Prontidao) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    if p.desligando.Load() {
        w.WriteHeader(http.StatusServiceUnavailable)
        return
    }
    ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
    defer cancel()
    if err := p.pool.Ping(ctx); err != nil {
        w.WriteHeader(http.StatusServiceUnavailable)
    }
}
```

```

    return
}
w.WriteHeader(http.StatusOK)
}

```

A divisão é a mesma do curso de .NET (Módulo 11.1): a liveness não olha o banco, para que uma queda dele não reinicie todas as réplicas; a readiness olha, para tirar do tráfego a réplica que não consegue trabalhar. O `atomic.Bool` existe porque o desligamento e as sondas rodam em goroutines diferentes. As rotas de saúde ficam fora do grupo `/v1` (Módulo 5.4), sem autenticação nem rate limiting.

11.3 Logs estruturados com slog

O `log/slog` é o logger estruturado da biblioteca padrão. Com o handler JSON, cada registro vira uma linha JSON que qualquer coletor entende. O que falta é o `trace_id` em cada linha, para pular de um log para o trace da requisição. Um handler que embrulha outro resolve:

```

type comTrace struct{ slog.Handler }

func (h comTrace) Handle(ctx context.Context, r slog.Record) error {
    if sc := trace.SpanContextFromContext(ctx); sc.IsValid() {
        r.AddAttrs(slog.String("trace_id", sc.TraceID().String()),
slog.String("span_id", sc.SpanID().String()))
    }
    return h.Handler.Handle(ctx, r)
}

func (h comTrace) WithAttrs(a []slog.Attr) slog.Handler { return
comTrace{h.Handler.WithAttrs(a)} }
func (h comTrace) WithGroup(n string) slog.Handler      { return
comTrace{h.Handler.WithGroup(n)} }

log := slog.New(comTrace{slog.NewJSONHandler(os.Stdout,
&slog.HandlerOptions{Level: nivel}}))

```

`WithAttrs` e `WithGroup` precisam ser reescritos: sem eles, um `log.With(...)` devolveria o handler interno e perderia o `trace_id`. E o trace só aparece nas chamadas que recebem o contexto (`log.InfoContext(ctx, ...)`, `slog.ErrorContext`), por isso o curso usa sempre as variantes `*Context`. O mesmo tipo de embrulho serve para mascarar dados pessoais (LGPD) num lugar só, com `ReplaceAttr`. Desde o Go 1.26, `slog.NewMultiHandler` envia o mesmo registro para vários destinos, útil durante uma migração de coletor.

11.4 OpenTelemetry

A instrumentação em Go é explícita: você configura os provedores uma vez e liga cada biblioteca a eles.

```
func iniciarTelemetria(ctx context.Context) (func(context.Context) error,
error) {
    recurso := resource.NewSchemaless(semconv.ServiceName("cobranca-api"))

    expTraces, err := otlptracegrpc.New(ctx) // endpoint via
OTEL_EXPORTER_OTLP_ENDPOINT
    if err != nil {
        return nil, err
    }
    traces := sdktrace.NewTracerProvider(sdktrace.WithBatcher(expTraces),
sdktrace.WithResource(recurso))

    expMetricas, err := otelmetricgrpc.New(ctx)
    if err != nil {
        return nil, err
    }
    metricas := sdkmetric.NewMeterProvider(
        sdkmetric.WithReader(sdkmetric.NewPeriodicReader(expMetricas)),
sdkmetric.WithResource(recurso))

    otel.SetTracerProvider(traces)
    otel.SetMeterProvider(metricas)
    otel.SetTextMapPropagator(propagation.TraceContext{}) // lê e propaga o
header traceparent

    return func(ctx context.Context) error {
        return errors.Join(traces.Shutdown(ctx), metricas.Shutdown(ctx)) //
descarrega o que está em buffer
    }, nil
}
```

Com os provedores globais configurados:

- **HTTP de entrada:** o middleware `otelhttp` (já na cadeia do Módulo 5.4) cria um span por requisição e emite o histograma `http.server.request.duration`, com a rota como dimensão. Versões recentes usam o `r.Pattern` do mux para isso, o que mantém a cardinalidade baixa.
- **HTTP de saída:** `otelhttp.NewTransport` no `http.Client` compartilhado.
- **Banco:** um tracer para o pgx (por exemplo, o pacote `otelpgx`) configurado em `cfgPool.ConnConfig.Tracer` cria um span por consulta.

- **Runtime:** o pacote de instrumentação de runtime do OpenTelemetry exporta memória, GC e goroutines.
- **Métricas de negócio:** `otel.Meter("cobranca")` cria contadores e histogramas, com as mesmas regras de cardinalidade do curso de arquivos (Módulo 9).

A função de desligamento devolvida por `iniciarTelemetria` vai num `defer` do `run`, com um contexto de alguns segundos: sem ela, os últimos spans antes do desligamento se perdem.

11.5 A imagem de container

Um serviço Go sem cgo compila para um binário estático que não precisa de nada do sistema operacional. A imagem final pode ser praticamente só ele:

```
FROM golang:1.27 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" -o /out/api ./cmd/api

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/api /api
USER nonroot:nonroot
ENTRYPOINT ["/api"]
```

O resultado tem poucas dezenas de MB, roda como usuário não-root, não tem shell nem gerenciador de pacotes (menos superfície de ataque) e sobe em milissegundos. A imagem *distroless static* inclui os certificados de autoridade e dados de fuso horário que a API precisa para chamar HTTPS e calcular a data de negócio em São Paulo. A versão do código (commit, data) já fica embutida no binário pelo `go build` e pode ser lida com `debug.ReadBuildInfo()` para expor num endpoint de versão ou como atributo do recurso do OpenTelemetry.

11.6 No Kubernetes

Os pontos específicos de Go, além dos do curso de .NET (sondas, autoscaling por CPU ou por requisições, limites do ingress para upload e SSE, forwarded headers):

- **Memória:** defina `GOMEMLIMIT` um pouco abaixo do limite do pod (por exemplo, `900MiB` para um limite de `1Gi`). Sem ele, o GC não sabe que existe um teto e o pod pode ser morto por OOM num pico.
- **CPU:** nada a fazer; o `GOMAXPROCS` segue o limite do cgroup desde o Go 1.25.

- **Desligamento:** `terminationGracePeriodSeconds: 40`, coerente com os 5 + 25 segundos do 11.1.
- **pprof:** a porta de administração (Módulo 9.1) não é exposta no Service; acesse com `kubectl port-forward` quando precisar.

11.7 Diagnóstico

O painel e a ordem de investigação são os do curso de .NET (Módulo 11.7): requisições por segundo, erros por classe, p95/p99 por rota, saturação e dependências. A saturação em Go se lê em três sinais: conexões em uso do pgxpool perto de `MaxConns` (o banco é o gargalo), número de goroutines crescendo sem parar (vazamento, confirme no perfil `goroutineleak`) e uso de CPU alto com o GC no topo do flame graph (alocação demais num caminho quente).

Exercício do Módulo 11

Implemente `servir`, `Prontidao` e `iniciarTelemetry` e suba o Aspire Dashboard em Docker como coletor OTLP. Rode um teste de carga e, no meio, mande SIGTERM ao processo: confirme que `/health/ready` passou a 503, que nenhuma requisição em andamento falhou e que um stream de progresso aberto foi encerrado e reconectado. Provoque um erro 500 e encontre o trace dele a partir do `traceId` do `ProblemDetails` e do `trace_id` da linha de log. Por fim, construa a imagem e confira o tamanho com `docker images`.

Projeto final — a API de cobrança em Go, passo a passo

O projeto final é a mesma API do curso de .NET, com os mesmos endpoints e o mesmo contrato de erros, construída em dez etapas com critério verificável por teste. Ela lê e grava as tabelas do pipeline de arquivos do primeiro curso: uma API em Go e workers em .NET sobre o mesmo PostgreSQL.

Estrutura do módulo

```

cobranca-api/
├─ go.mod                                go 1.27; ferramentas (sqlc, oapi-codegen)
como "tool"
├─ cmd/api/main.go                      main + run
├─ api/
│   ├─ openapi.yaml                    o contrato
│   └─ oapi-codegen.yaml
├─ internal/
│   ├─ app/                            Dependências e MontarHandler
│   ├─ api/                            código gerado do contrato
│   ├─ arquivos/  titulos/  ocorrencias/
│   ├─ dominio/
│   └─ banco/                          queries.sql, sqlc.yaml, código gerado,
migrations
│   └─ plataforma/                     httpx, problema, validacao, auth, limites,
saude, telemetria
├─ tests/                              integração (TestMain com containers)
└─ deploy/
    ├─ Dockerfile
    ├─ docker-compose.yml              PostgreSQL, Keycloak, Aspire Dashboard
    └─ k8s/

```

As migrations são as mesmas do curso de arquivos (tabelas `arquivo`, `retorno_detalhe`, `outbox`, `idempotencia`). Se os workers .NET já as aplicam com EF Core, a API em Go só as lê para o `sqlc generate`; defina um único dono das migrations para não haver duas fontes da verdade do schema.

O run final

```

func run(ctx context.Context, getenv func(string) string, saida io.Writer)
error {
    cfg, err := carregarConfig(getenv)
    if err != nil {
        return fmt.Errorf("configuração: %w", err)
    }
    log := slog.New(telemetria.ComTrace(slog.NewJSONHandler(saida, nil))) //
Módulo 11.3
    slog.SetDefault(log)

    desligarTelemetria, err := telemetria.Iniciar(ctx) // Módulo 11.4
    if err != nil {
        return fmt.Errorf("telemetria: %w", err)
    }
}

```

```

    defer func() {
        ctxFim, cancel := context.WithTimeout(context.Background(),
5*time.Second)
        defer cancel()
        _ = desligarTelemetria(ctxFim)
    }()

    pool, err := banco.NovoPool(ctx, cfg.BancoURL) // MaxConns + tracer do
pgx (Módulos 6.1 e 11.4)
    if err != nil {
        return fmt.Errorf("banco: %w", err)
    }
    defer pool.Close()

    validador, err := auth.NovoValidador(ctx, cfg.JWTEmissor,
cfg.JWTAudencia) // Módulo 8.1
    if err != nil {
        return fmt.Errorf("jwt: %w", err)
    }

    srv := &http.Server{ // Módulo 2.1
        Addr:            cfg.Endereco,
        ReadHeaderTimeout: 5 * time.Second,
        ReadTimeout:      30 * time.Second,
        WriteTimeout:      30 * time.Second,
        IdleTimeout:       120 * time.Second,
        MaxHeaderBytes:    64 << 10,
    }
    desligando := make(chan struct{})
    srv.RegisterOnShutdown(func() { close(desligando) }) // Módulo 11.1

    prontidao := saude.NovaProntidao(pool)
    srv.Handler = app.MontarHandler(app.Dependencias{
        Pool:            pool,
        Storage:          armazenamento.NovoS3(cfg.Bucket),
        Verificador:      validador,
        Log:              log,
        Prontidao:        prontidao,
        Desligando:       desligando,
    })

    go servirAdmin(ctx) // pprof em 127.0.0.1:6060 (Módulo 9.1)
    return servir(ctx, srv, prontidao) // Módulo 11.1
}

```

MontarHandler: o mesmo grafo em produção e nos testes

```
func MontarHandler(d Dependencias) http.Handler {
    consultas := banco.New(d.Pool) // sqlc (Módulo 6.2)

    arquivosH := arquivos.NovosHandlers(
        arquivos.NovaConsulta(consultas),
        arquivos.NovoRecebimento(d.Pool, d.Storage),
        d.Desligando, // o handler de progresso encerra os streams no
desligamento
    )
    titulosH :=
titulos.NovosHandlers(titulos.NovoCache(titulos.NovaConsulta(consultas),
time.Minute)) // Módulo 9.5
    ocorrenciasH := ocorrencias.NovosHandlers(d.Pool)

    mux := http.NewServeMux()
    v1 := httpx.NovoGrupo(mux, "/v1",
        auth.Autenticar(d.Verificador), // 1. quem é (Módulo 8.2)
        limites.PorCliente(100),        // 2. quantas vezes (Módulo 8.4)
    ) // o prazo de 10 s fica em cada rota comum, dentro de cada Registrar
(Módulo 9.4)
    arquivosH.Registrar(v1)
    titulosH.Registrar(v1)
    ocorrenciasH.Registrar(v1)

    mux.Handle("GET /health/live", saude.Vivo())
    mux.Handle("GET /health/ready", d.Prontidao)
    mux.HandleFunc("GET /openapi.yaml", api.ServirEspecificacao) // Módulo
7.6

    return httpx.Encadear(mux,
        httpx.ComRecuperacao(d.Log), // Módulo 4.6: o mais externo
        otelhttp.NewMiddleware("cobranca-api"), // Módulo 11.4: span e
métricas por requisição
        httpx.ComLog(d.Log), // Módulo 2.3: já dentro do
span, com trace_id
    )
}
```

Quem abre só este arquivo vê a API inteira: as três features, o que protege `/v1`, o que fica de fora e a ordem dos middlewares globais. A struct `Dependencias` do Módulo 10 ganhou `Prontidao` e `Desligando`; nos testes, eles são uma prontidão real sobre o pool do container e um canal que nunca fecha.

As dez etapas

- ☐ **1. Esqueleto.** Módulo Go, `docker-compose`, migrations aplicadas, `run` com `carregarConfig`, `servir` e os health checks. Critério: `/health/ready` responde 200 com o banco e 503 sem ele; SIGTERM desliga sem erro.
- ☐ **2. Primeiro endpoint.** `GET /v1/arquivos/{id}` com `sqlc`, `httpx.Handler`, `ProblemDetails` e o `TestMain` com container. Critério: testes de integração para 200, 404 e UUID inválido (400).
- ☐ **3. Autenticação e autorização.** Autenticar com go-oidc e Keycloak, `ExigirEscopo`, `exigirDono`. Critério: testes de 401, 403 e 404 para arquivo de outro cliente (com o verificador falso), e um teste manual com token real.
- ☐ **4. Listagem.** `lerFiltro`, cursor keyset, `ListarArquivos` com `sqlc.narg`. Critério: 400 com `errors` para filtros inválidos; páginas sem repetição; plano com índice.
- ☐ **5. Upload.** Corpo cru com `MaxBytesReader`, prazo de leitura por rota, hash com `TeeReader`, `ON CONFLICT` no hash, outbox. Critério: 400 MB com memória estável no perfil de heap; duplicado devolve 409 com `arquivoExistenteId`.
- ☐ **6. Idempotência.** Tabela e middleware de 6.6. Critério: mesma chave, mesma resposta; chave reutilizada com outro conteúdo, 422.
- ☐ **7. Títulos e exportação.** `NossoNumero`, `CacheDeTítulos` com `singleflight`, exportação CSV em streaming com `ErrAbortHandler`. Critério: 1 milhão de linhas com memória estável; falha no meio corta a conexão.
- ☐ **8. Progresso em tempo real.** SSE com `ResponseController.Flush`, ping e encerramento no desligamento. Critério: o navegador acompanha um arquivo processado pelo worker .NET; um SIGTERM encerra o stream e o cliente reconecta.
- ☐ **9. Contrato.** `openapi.yaml` completo, servidor strict para os endpoints JSON, validação de respostas nos testes, `oasdiff` no CI. Critério: remover um campo do contrato quebra o build.
- ☐ **10. Produção.** Rate limiting, telemetria, imagem distroless, deploy no Kubernetes com `GOMEMLIMIT` e sondas. Critério: p95 de consultas abaixo de 200 ms com 50 usuários virtuais, e um rollout durante o teste sem erros 5xx.

Critérios de aceite finais

- ☐ Toda resposta de erro é `ProblemDetails` com `type` estável e `traceId`, no mesmo formato da API em .NET
- ☐ Nenhuma rota de negócio existe fora do grupo autenticado (e um teste quebra se existir)
- ☐ Nenhum cliente lê ou altera dados de outro, com teste para cada rota com ID
- ☐ O código gerado do contrato está em dia e o CI detecta mudanças incompatíveis

- ☐ Upload, listagem e exportação usam memória constante
- ☐ Repetir um `POST` com a mesma `Idempotency-Key` nunca produz efeito duplicado
- ☐ `go test -race ./...` passa limpo, com integração contra PostgreSQL real em menos de um minuto
- ☐ `govulncheck ./...` limpo e o perfil `goroutineleak` vazio depois do teste de carga
- ☐ O desligamento não derruba requisições em andamento

Referências e próximos passos

Com este curso, a API de cobrança existe em duas plataformas sobre o mesmo banco e o mesmo contrato de erros. Comparar as duas implementações lado a lado, módulo a módulo, é o melhor exercício de arquitetura que os três cursos oferecem: os problemas são idênticos, e as diferenças nas soluções mostram o que é essencial e o que é convenção de cada ecossistema. Os próximos passos naturais são gRPC entre serviços internos (Go e .NET têm suporte de primeira linha) e consumir os eventos da outbox numa terceira aplicação.

Os outros dois cursos

- [Curso: .NET 10 e System Design para processamento de arquivos em alto volume](#): o pipeline que processa os arquivos que esta API recebe.
- [Curso: Minimal APIs no ASP.NET Core 10](#): a mesma API em .NET, com os diagramas de binding, metadados, token bucket e idempotência que valem para os dois.

Documentação por módulo

Módulo	Leitura
2. Arquitetura	Pacote net/http · Roteamento no ServeMux do Go 1.22
3. Fundamentos	encoding/json/v2 · Pacote uuid
6. Dados	pgx · sqlc
7. Contrato	oapi-codegen
8. Segurança	go-oidc · golang.org/x/time/rate · OWASP API Security Top 10
9. Performance	Diagnósticos em Go · errgroup e singleflight
10. Testes	testing/synctest · Fuzzing · testcontainers-go

Módulo	Leitura
11. Operação	log/slog · OpenTelemetry para Go

Livros

- *Let's Go Further*, Alex Edwards: uma API JSON completa com a biblioteca padrão, do roteamento ao deploy. É o complemento mais próximo deste curso.
- *100 Go Mistakes and How to Avoid Them*, Teiva Harsanyi: os erros de concorrência, contexto, interfaces e tratamento de erros que este curso tenta evitar, um por um.
- *Designing Data-Intensive Applications*, Martin Kleppmann: a mesma base teórica recomendada no curso de arquivos.

Fontes sobre versões

- [Go 1.27 Release Notes](#): lançado em agosto de 2026; métodos genéricos, `encoding/json/v2` e `jsoncontext`, o novo pacote `uuid`, o perfil `goroutineleak` disponível por padrão, `httptest.NewTestServer` e as mudanças do `net/http`.
- [Go 1.26 Release Notes](#): `new` com expressão, coletor Green Tea por padrão, `errors.AsType`, `slog.NewMultiHandler` e a causa do sinal em `signal.NotifyContext`.
- [Documentação do pacote uuid](#): `New`, `NewV4`, `NewV7` e `Parse` na versão 1.27.1.